

Web开发

技术基础

互联网和万维网

互联网（Internet）

定义：是由网络与网络互联而成的庞大网络，这些网络以标准的TCP/IP协议族相连，连接全世界数十亿个设备，形成逻辑上的单一巨大国际网络。它是由从本地到全球范围内几百万个私人的、学术界的、企业的和政府的网络所构成，通过电子、无线和光纤网络技术等一系列广泛的技术连接在一起。

功能：互联网承载了范围广泛的信息资源与服务，例如万维网(World Wide Web)、电子邮件、点对点网络、文件共享、在线网络游戏以及IP电话服务等。

万维网发展历史

万维网：World Wide Web/WWW/Web，一个由许多互相链接的超文本组成的系统，通过互联网访问。信息资源以页面（网页/Web页）的形式存储在服务器（Web站点）上，信息采用超文本（HyperText）方式进行组织，通过超链接将一页信息接到另一页信息，页面到页面的链接信息由统一资源定位符URL进行维持。

超文本：**泰德·尼尔森**预见并启发了万维网，他于1965年创造了“超链接Hypertext”和“超媒体Hypermedia”这两个术语，它体现了链接包括文本、音频和视频在内的对象网络的思想。

发明：英国科学家**蒂姆·伯纳斯-李**（Tim Berners-Lee）于1989年在欧洲核子研究组织（CERN）发明万维网。

到1990年12月25日，伯纳斯-李完成了Web系统工作 所需的所有工具：

- HyperText Transfer Protocol（HTTP）0.9
- HyperText Markup Language（HTML）第一个Web浏览（称为 WorldWideWeb, 也是Web编辑器）
- 第一个 HTTP server software，后来被称为CERN httpd
- 第一个Web服务器（<http://info.cern.ch>）
- 第一个描述项目的 Web页面

W3C创建者：伯纳斯-李

我国最早的：

- 电子邮件：1987年9月20日20点55分，兵器工业计算所钱天白研究员从北京向海外发出的电子邮件，成为中国用互联网向世界发出的第一封电子邮件。该邮件经意大利到达德国的卡尔斯鲁厄大学。

- WWW网站：1994年5月，中科院高能物理所网络正式连入Internet，并建立了中国的一个WWW网站。

万维网现在与未来

如今的互联网：

- 组成：
 - 表层网：4%左右
 - 深网（Deep Web）：96%左右，由无法通过谷歌等常规搜索引擎找到的数据组成，包括所有搜索引擎无法找到的Web页面，比如用户数据库、需要注册的在线论坛、Web邮箱页面、需要付费登陆的页面等
 - 暗网（Dark Web）：只能通过特殊软件、授权或对电脑作特别设置才能访问，在流行的搜索引擎上无法查到的特殊网络
- 长尾效应失效，数据孤岛爆发，走向割裂
- 三种病：虚假信息误导、数据隐私泄露、平台垄断造成的数据孤岛
- 梅特卡夫定律：一个网络的价值等于该网络内的节点数的平方，也就是说，网络的规模越大，其总价值也就越大。
- 割裂标志性事件：
 - 俄罗斯断网演习。2019年5月1日，俄罗斯总统普京签署了有关保障本国互联网稳定运行的《主权互联网法》。该法律于2019年11月1日正式生效，要求俄罗斯在国内建设一套独立于国际互联网的网络基础设施，确保俄方在遭遇外部“断网”等冲击时仍能稳定运行。最为严格和全面的一次：2023年7月4日到5日。
 - 美国“清洁网络”计划

• **清洁运营商**：确保中国移动等不受美国信任的中国电信公司不为美国和其他国家提供**国际电信服务**

• **清洁应用商店**：从美国**应用商店**中删除他们声称不信任的中国软件

• **清洁APP**：阻止华为和不受信任的手机供应商预先安装或下载美国最常用的**应用**软件；

• **清洁云服务**：限制阿里、腾讯等**中国云端服务提供商**在美国收集、存储和处理大量数据和敏感信息的能力

• **清洁电缆**：排斥中国公司竞标、建设和使用连接全球互联网的**海底电缆**

• **清洁路径**：督促美国盟国不使用华为和中兴通讯之类不可靠卖家提供的任何**5G设备**的倡议

Web基本结构和工作过程

Web基本**架构**：Web服务器（服务器端）、Web浏览器（客户端）、传输协议（HTTP协议）

基本**工作过程**：用户通过Web客户端应用程序（即浏览器）向Web服务器发出请求，请求通过HTTP协议送到Web服务器；Web服务器根据客户端请求将保存在Web服务器中的某个页面通过HTTP协议返回给Web客户端；浏览器接收到页面后对其进行解释，最终将图文并茂的画面呈现给用户

Web浏览器

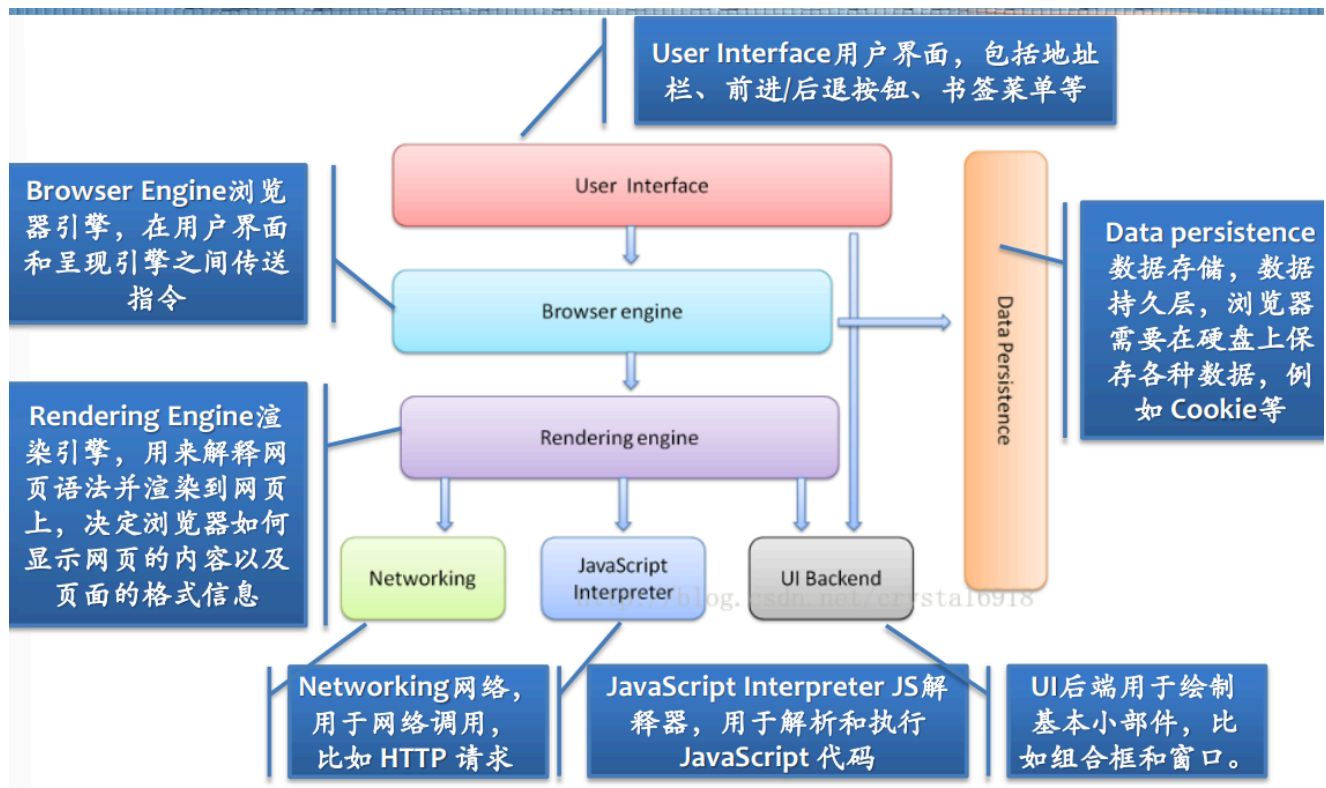
定义：Web客户端的程序

作用：浏览WWW服务器中的Web页面

工作过程：

- 接收用户的请求（键盘或鼠标输入）
- 利用HTTP协议将用户的请求传送给Web服务器
- 接收Web服务器送回的Web页面，并将其解释和显示

结构：



无头浏览器：无头浏览器是一种没有图形用户界面的web浏览器。这样的web浏览器可以使用命令行界面执行某些命令。可以用于：

- 测试基本流程和可选流程
- 模拟单击链接和按钮
- 自动填写和提交表格
- 测试SSL性能
- 尝试不同的服务器负载
- 获取关于页面响应时间的报告
- 获取有用的网站代码
- 截屏查看结果

Selenium：一个Web的自动化测试工具，最初是为网站自动化测试而开发的。可以直接运行在浏览器上，它支持所有主流的浏览器，包括PhantomJS这些无界面的浏览器

PhantomJS：一个基于Webkit的无界面（headless）浏览器，它会把网站加载到内存并执行页面上的JavaScript，因为不会展示图形界面，所以运行起来比完整的浏览器要高效

Web服务器

Web服务器可以分布在Internet的任意位置，每个Web服务器都保存着可以被Web浏览器共享的信息，这些信息通常以页面（Web页面）的方式进行组织，页面一般是**超文本/超媒体**文档，页面间通过**超链接**建立连接。应实现**HTTP协议功能**，接收和处理浏览器的请求。

超链接：超链接指向的资源可以处于Internet的任一Web服务器之中，利用超链接Web页面可以与其他Web页面、多媒体信息进行关联。

超文本与超媒体：

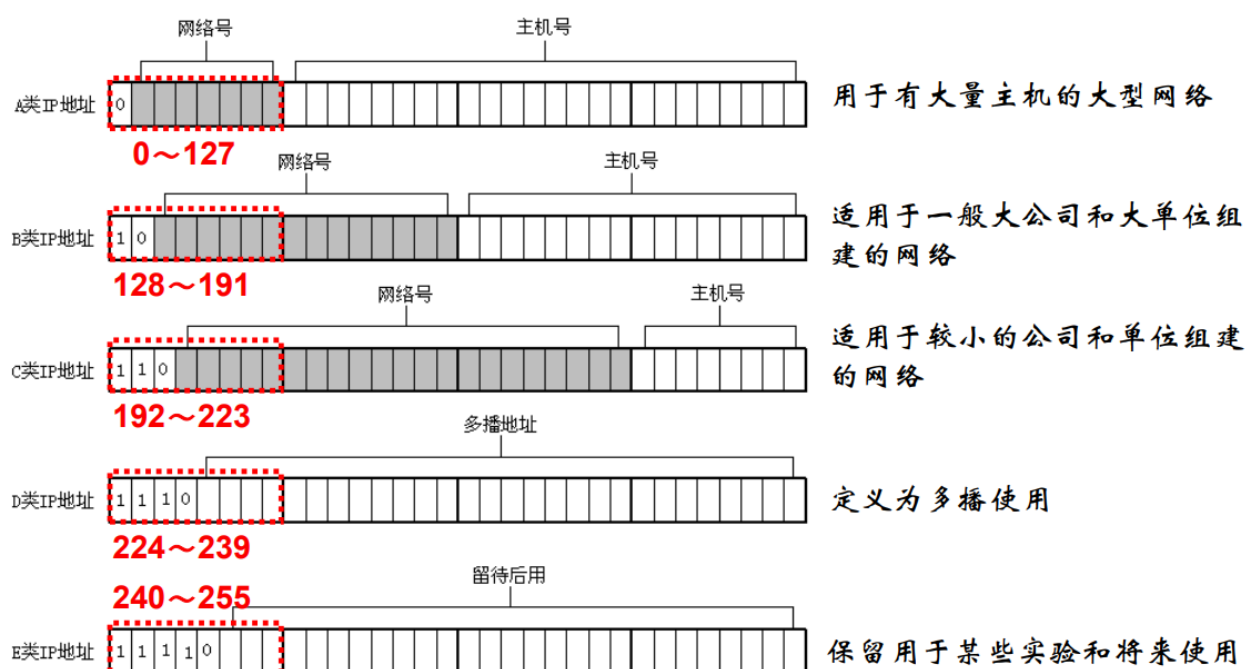
- 万维网是分布式超媒体（hypermedia）系统，它是超文本（hypertext）系统的扩充。
- 一个超文本由多个信息源链接成。利用一个链接可使用户找到另一个文档。这些文档可以位于世界上任何一个接在Internet上的超文本系统中。**超文本是万维网的基础**。
- 超媒体与超文本的区别是**文档内容不同**。超文本文档仅包含文本信息，而超媒体文档还包含其他表示方式的信息，如图形、图像、声音、动画，甚至活动视频图像。

C10X问题：包括C10K、C10M等，其核心思想是让服务器处理更多的请求，追求单机极致性能（当前面临的是C10M，指单机同时处理一千万的请求）

IP与域名寻址

IP地址

- 定义：给每个连接在Internet上的主机或路由器分配一个在全世界范围是唯一的**32 bit**的标识符，采用**点分十进制**表示
- 构成：在IPv4协议中，每个IP地址由两部分组成：网络号（Netid）和主机号（Hostid）。网络号用于标识一个网络；主机号用于标识在该网络中的一个主机。
- 分类：



- 私有网IP：保留给私有用户，不必全球唯一。可以用于未连到Internet的网络，也可以用于采用NAT技术连到Internet的网络。

- A类NetID: 10.0.0
- B类NetID: 172.16~172.31
- C类NetID: 192.168.0~192.168.255

域名系统 (DNS)

- 定义: Internet/Intranet 用于提供域名登记和域名到IP地址转换的一组服务
- 作用: 在IP地址和具有实际意义的计算机名称之间建立一种对应关系, 从而使用户可以使用有实际意义的名称而不是难记的IP地址来访问其他计算机
- 组成:
 - **DNS域名空间**: Internet上所有主机的唯一和比较友好的主机名所组成的空间, 是DNS命名系统在一个层次上的逻辑树结构;
 - **DNS服务器**: 运行DNS服务器程序的计算机, 负责存储记录和域名解析请求;
 - **DNS客户端**: 也称为解析器, 通常内置在实用程序中 (或通过库函数访问), 用于向DNS服务器提出域名解析请求并处理DNS服务器的反馈信息;
 - **资源记录**: DNS数据库中的信息集, 每台DNS服务器都所需资源记录, 用来回答DNS名字空间的查询。
- 域名结构:
 - 根域: 根域DNS服务器只负责处理顶级域名的解析请求
 - 顶级域: 顶级域有两种划分方式, 一种是以**行业组织**的形式来划分, 一种是以**国别或地区**的方式来划分
 - 二级域: 通常用来表示某一具体的**公司或企业**, 必须先注册才能使用
 - 子域: 子域是企业或单位根据自己的需要对二级域的进一步划分, 这些名称必须从已注册的二级域名中派生。一个子域下面可以继续划分子域, 或者接挂主机
 - 主机: 最下一层是主机或资源名称, 由子域管理员自行建立
- 域名解析:
 - 一种进行域名与IP地址之间的映射的机制, 将域名映射为对应的IP地址 (或将IP地址映射为对应的域名), 采用一个联机分布式数据库系统, 利用客户/服务器模式进行工作, 需要借助于一组既相互独立又相互协作的域名服务器完成。
 - 模式: 迭代查询/递归查询
 - 工具: `dig @server name type`
 - `@server`: 指定域名服务器
 - `name`: 指定查询请求资源的域名
 - `type`: 指定查询类型, 默认为A
- 问题: 保密性 (隐私风险)、完整性 (DNS欺骗)
- 防护协议:
 - **DNSSEC**: 在DNS协议的基础上增加了一个数字签名机制, 实现完整性保护、没有考虑保密性, 下层只支持UDP;
 - **DNSCrypt**: 域名的查询请求与响应结果都是加密的, 实现了完整性和保密性, 下层支持TCP和UDP, 但是未提交过RFC未标准化, 易被监听者识别;

- **DoT (DNS over TLS)**: 基于TLS来进行报文加密的DNS请求交互协议，侧重于DNS交互报文的保密性，标准化，使用TCP端口853完成TLS握手，再执行普通的DNS请求/应答，因此执行过程中将使用至少4次RTT导致DNS 的查询延时放大4倍，无法被单独识别（完全包裹在TLS里）；
- **DoH (DNS over HTTPS)**: DoH 相当于双重隧道的协议，最终也是依靠TLS来实现了保密性与完整性，使用HTTPS通用端口443，更难以检测，容易开发客户端，但是域名解析耗时较原DNS协议会显著增加。

协议类型	标准化	完整性	保密性	抗协议识别	主流的浏览器支持	知名的公共服务器
DNSSEC	有 RFC 4033, RFC 4034, and RFC 4035	有	无	无	无	Google Cloudflare Quad9
DNSCrypt	无	有	有	无	无	OpenDNS Quad9
DNS over TLS	有 RFC 7858	有	有	有	有	Cloudflare Quad9
DNS over HTTPS	有 RFC 8484	有	有	有	有	Google Cloudflare Quad9

统一资源定位符URL

所有网页在 Internet 上统一的地址形式，是对可以从因特网上得到的资源的位置和访问方法的一种简洁的表示，也被称为网页地址。它最初是由蒂姆·伯纳斯·李发明用来作为万维网的地址的，现在已被万维网联盟编制为因特网标准RFC1738。URL给资源的位置提供一种抽象的识别方法，并用这种方法给资源定位。

构成：

- 协议：也称为服务协议或者服务方式。例如http://，mailto，ftp://等；
- 主机名：是指存放资源的服务器所使用的域名或IP地址，例如域名www.example.com或者其对应的IP地址；
- 端口号：是访问Internet上的一台计算机的某种资源时，受访问计算机与外界进行数据通讯的出口，以数字方式表示，若为HTTP的默认值“:80”可省略；
- 路径：指资源或者信息在服务器上存放的位置，通常由“目录/子目录/”这样的结构组成；
- 文件名：指资源或者信息的文件名，即一个网页的名字。

格式（字符对大小写没有要求）：

- **标准**： 协议类型:[//服务器地址[:端口号]][/资源层级UNIX文件路径]文件名[?查询][#片段ID]
- **完整**： 协议类型:[//[访问资源需要的凭证信息@]服务器地址[:端口号]][/资源层级UNIX文件路径]文件名[?查询][#片段ID]

- URI：用于**标识**某个资源的字符串。它是一个抽象或标准的名称，用来区分互联网上的每一个资源；
- URL：不仅**标识**资源，还提供**定位**该资源（即如何访问它）所需的信息；
- URN：仅用于**唯一命名**一个资源，而不提供定位信息，它的命名是**持久且全局唯一**的。

URL和域名的主要区别在于，URL是一个字符串，提供网页的信息位置或完整的互联网地址，而域名是URL的一部分，它是一个更人性化的IP地址形式。

超文本传送协议HTTP

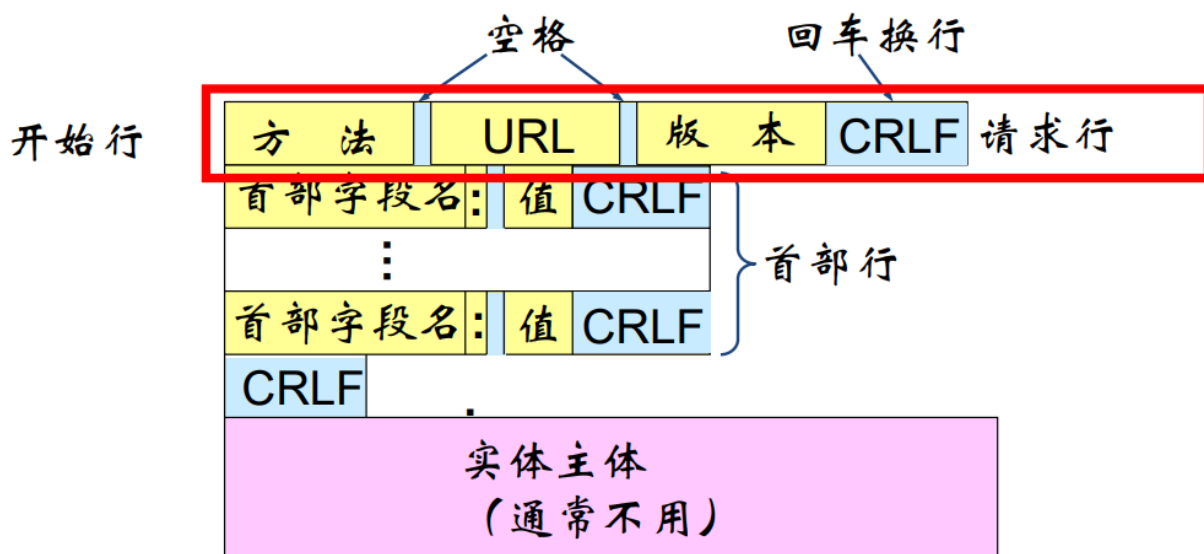
HTTP：超文本传送协议HTTP（HyperText Transfer Protocol）是Web客户机与Web服务器之间的传输协议。它建立在TCP的基础上，是一种面向事务的（transaction-oriented）应用层协议，它是万维网上能够可靠地交换文件（包括文本、声音、图像等）的重要基础。

特点：

- **面向事务**的客户服务器协议（事务：一系列的信息交换，而这一系列的信息交换是一个不可分割的整体）即：要么所有的信息交换都完成，要么一次交换都不进行。
- HTTP 1.0协议是**无状态**的：
 - 服务器不记得曾经访问过的客户，也不记得曾经服务过多少次
 - Cookies用于记录用户
- HTTP协议本身是**无连接**的
 - HTTP底层使用了面向连接的TCP向上提供的服务
 - HTTP可以基于无连接协议UDP

报文：

- 请求报文：从客户向服务器发送请求报文

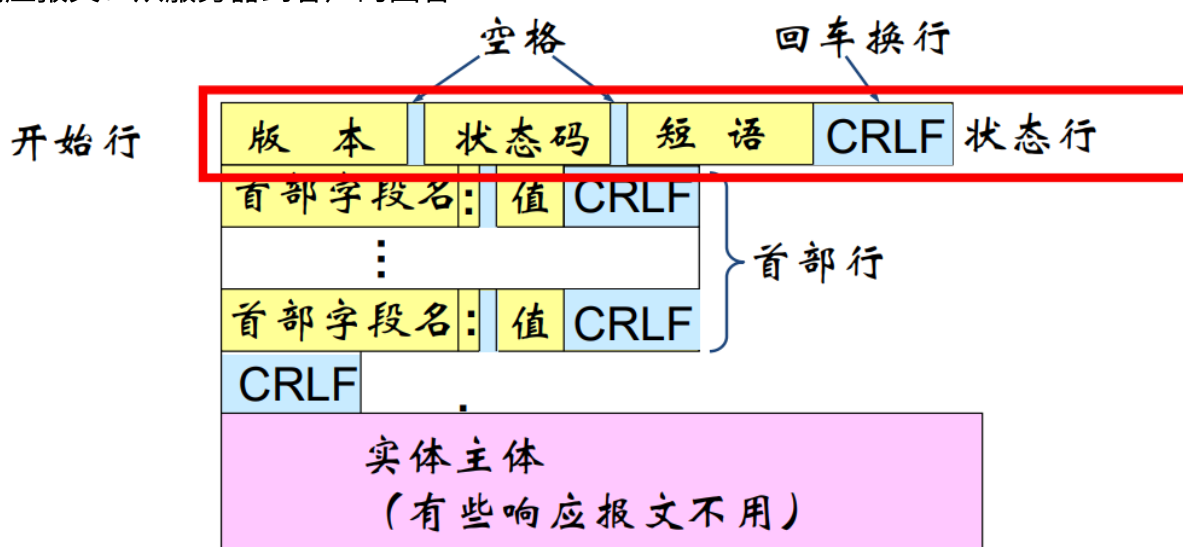


报文由三个部分组成，即**开始行**、**首部行**和**实体主体**。在请求报文中，开始行就是**请求行**

- 方法：对所请求的对象进行的操作，实际上就是一些命令。请求报文的类型是由它所采用的方法决定的。

方法（操作）	意 义
OPTION	请求一些选项的信息
GET	请求读取由URL所标志的信息
HEAD	请求读取由URL所标志的信息的首部
POST	给服务器添加信息（例如，注释）
PUT	在指明的URL下存储一个文档
DELETE	删除指明的URL所标志的资源
TRACE	用来进行环回测试的请求报文
CONNECT	用于代理服务器

- URL：所请求的资源的 URL。
- 版本：HTTP的版本
- 响应报文：从服务器到客户的回答



报文由三个部分组成，即**开始行**、**首部行**和**实体主体**。在响应报文中，开始行就是**状态行**

- 状态码：
 - 1xx：表示通知信息，如请求收到了或正在进行处理
 - 2xx：表示成功，如接受或知道了
 - 3xx：表示重定向，表示要完成请求还必须采取进一步的行动
 - 4xx：表示客户的差错，如请求中有错误的语法
 - 5xx：表示服务器的差错，如服务器失效无法完成请求
- 短语：解释状态码的简单短语
- 常见：
 - **200 OK**：客户端请求成功
 - **301 Moved permanently**：HTTP 301 永久重定向说明请求的资源已经被移动到了由 Location 头部指定的url上，是固定的不会再改变。搜索引擎应根据该响

应修正

- **302 Found**：HTTP 302 重定向状态码表明请求的资源被暂时的移动到了由 Location 头部指定的 URL 上。浏览器会重定向到这个URL，但是搜索引擎不会对该资源的链接进行更新
- **304 Not modified**：HTTP 304 未改变说明无需再次传输请求的内容，也就是说可以使用缓存的内容。
- **400 Bad Request**：客户端请求有语法错误，不能被服务器所理解
- **401 Unauthorized**：请求未经授权，这个状态代码必须和WWW-Authenticate 报头域一起使用
- **403 Forbidden**：服务器收到请求，但是拒绝提供服务
- **404 Not Found**：请求资源不存在，eg：输入了错误的URL
- **500 Internal Server Error**：服务器发生不可预期的错误
- **503 Server Unavailable**：服务器当前不能处理客户端的请求，一段时间后可能恢复正常

请求头字段名	说明
Accept	能够接受的回应内容类型（Content-Types）。
Accept-Charset	能够接受的字符集
Accept-Datetime	能够接受的按照时间来表示的版本
Accept-Encoding	能够接受的编码方式列表。参考HTTP压缩。
Accept-Language	能够接受的回应内容的自然语言列表。
Authorization	用于超文本传输协议的认证的认证信息
Cache-Control	用来指定在这次的请求/响应链中的所有缓存机制 都必须 遵守的指令
Connection	该浏览器想要优先使用的连接类型
Content-Length	以 八位字节数组（8位的字节）表示的请求体的长度
Content-MD5	请求体的内容的二进制 MD5 散列值，以 Base64 编码的结果
Content-Type	请求体的 多媒体类型（用于POST和PUT请求中）
Cookie	之前由服务器通过 Set- Cookie（下文详述）发送的一个 超文本传输协议 Cookie
Date	发送该消息的日期和时间(按照 RFC 7231 中定义的"超文本传输协议日期"格式来发送)
Expect	表明客户端要求服务器做出特定的行为
From	发起此请求的用户的邮件地址
Host	服务器的域名(用于虚拟主机)，以及服务器所监听的传输控制协议端口号。如果所请求的端口是对应的服务的标准端口，则端口号可被省略。

请求头字段名	说明
If-Match	仅当客户端提供的实体与服务器上对应的实体相匹配时，才进行对应的操作。主要作用时，用作像 PUT 这样的方法中，仅当从用户上次更新某个资源以来，该资源未被修改的情况下，才更新该资源。
If-Modified-Since	允许在对应的内容未被修改的情况下返回304未修改（ 304 Not Modified ）
If-None-Match	允许在对应的内容未被修改的情况下返回304未修改（ 304 Not Modified ）
If-Range	如果该实体未被修改过，则向我发送我所缺少的那一个或多个部分；否则，发送整个新的实体
If-Unmodified-Since	仅当该实体自某个特定时间以来未被修改的情况下，才发送回应。
Max-Forwards	限制该消息可被代理及网关转发的次数。
Origin	发起一个针对 跨来源资源共享 的请求。
Pragma	与具体的实现相关，这些字段可能在请求/回应链中的任何时候产生多种效果。
Proxy-Authorization	用来向代理进行认证的认证信息。
Range	仅请求某个实体的一部分。字节偏移以0开始。参见字节服务。
Referer	表示浏览器所访问的前一个页面，正是那个页面上的某个链接将浏览器带到了当前所请求的这个页面。
TE	浏览器预期接受的传输编码方式：可使用回应协议头 Transfer-Encoding 字段中的值；
Upgrade	要求服务器升级到另一个协议。
User-Agent	浏览器的浏览器身份标识字符串
Via	向服务器告知，这个请求是由哪些代理发出的。
Warning	一个一般性的警告，告知，在实体内容体中可能存在错误。

发展历史（5个版本）

- HTTP/0.9：无状态，GET，纯文本
- HTTP/1.0：
 - 请求与响应支持头域
 - 响应对象以一个响应状态行开始
 - 响应对象不只限于超文本
 - 开始支持客户端通过POST方法向Web服务器提交数据，支持GET、HEAD、POST方法
 - 支持长连接（但默认还是使用短连接），缓存机制，以及身份认证
- HTTP/1.1：

- 引入了**持久连接**（persistent connection），即TCP连接默认不关闭，可以被多个请求复用，不用声明Connection: keep-alive
- 引入了**管道机制**（pipelining），即在同一个TCP连接里，客户端可以同时发送多个请求，进一步改进了HTTP协议的效率
- 新增方法：PUT、PATCH、OPTIONS、DELETE
- 缓存处理：Entity tag, If-Unmodified-Since, If-Match, If-None-Match
- 带宽优化和网络连接的使用：引入了range头域允许只请求资源的某个部分
- 支持Host头域：一台物理服务器上可以存在多个虚拟主机
- HTTP/2:
 - IETF RFC7540
 - 二进制协议，不再是纯文本，这是HTTP2性能加强的核心
 - 复用TCP连接，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，且不用按顺序一一对应，部分解决了“队头堵塞”，此双向的实时通信称为多工（Multiplexing）
 - 利用HPACK对消息头进行压缩传输，减少数据传输量
 - 引入服务端推送（Server Push）机制，允许服务器未经请求，主动向客户端发送资源
 - 引入ALPN协商机制，让客户端和服务端选择使用HTTP 1.1还是2
- HTTP/3: 基于 QUIC（Quick UDP Internet Connections），运行在 UDP 之上

Web技术的核心：

- URL：解决文档（资源）的命名和寻址
- HTTP：解决文档（资源）的快速传输
- HTML：解决超文本文档的表示

超文本标记语言HTML

概念

W3C和Web标准

W3C：

- 万维网联盟（World Wide Web Consortium）
- 创建于1994年10月
- 由 Tim Berners-Lee 创建
- 是一个国际性的标准组织
- 工作是对 Web 进行标准化
- 创建并维护 WWW 标准
- W3C 标准被称为 W3C 推荐（W3C Recommendations）

Web标准：一系列标准的集合，主要包括结构（Structure）、呈现（Presentation）和行为（Behavior）三个方面，相互分离

- 结构标准：
 - 决定网页的结构和内容，包括HTML、XML、XHTML
 - XML（The Extensible Markup Language）是一种可扩展标记语言。最初的目的是为了弥补HTML的不足，它具有强大的扩展性，可用于数据的转换和描述。
 - XHTML（The Extensible HyperText Markup Language）是可扩展超文本标识语言
 - HTML主要用于网页的结构和内容；XML则用于数据的存储和传输，强调数据的自描述性；而XHTML是HTML的一种严格版本，结合了XML的语法规则。简而言之，HTML更关注表现，XML更关注数据，而XHTML则是HTML的规范化形式。
- 呈现标准：CSS
 - 用于设置网页元素的版式、颜色、大小等外观样式
 - CSS（Cascading Style Sheet）是层叠样式表。CSS标准建立的目的是以CSS为基础进行网页布局，控制网页的呈现。CSS布局与HTML结构语言相结合，可以实现呈现与结构的分离，使网站的访问及维护更加容易
- 行为标准：DOM、ECMAScript
 - 行为是指网页模型的定义及交互的编写
 - DOM（Document Object Model）是文档对象模型。W3C文档对象模型（DOM）是中立于平台和语言的接口，它允许程序和脚本动态地访问和更新文档的内容、结构和样式
 - ECMAScript是欧洲计算机制造商协会（ECMA）国际以JavaScript为基础制定的标准脚本语言
- 优点：
 - 页面响应快：HTML文档体积变小，响应时间短
 - 易于维护：只需更改CSS文件，就可以改变整站的样式
 - 可访问性：语义化的HTML（结构和呈现相分离的HTML）编写的网页文件，更容易被屏幕阅读器识别
 - 设备兼容性：不同的样式表可以让网页在不同的设备上呈现不同的样式
 - 搜索引擎：语义化的HTML能更容易被搜索引擎解析，提升排名

HTML简介

HTML：HyperText Markup Language，超文本标记语言，一种用来结构化Web网页及其内容的标记语言。主要是通过HTML标签（HTML tag）对网页中的文本、图片、声音等内容进行描述，HTML提供了许多标签，如段落标签、标题标签、超链接标签、图片标签等，网页中需要定义什么内容，就用相应的HTML标签描述即可。

XHTML：针对HTML语法过于宽松问题定义的更严格更纯净的HTML，其本质是重新用XML来表示的HTML 4。（XHTML2兼容问题，工作停止）它与HTML 4有六个主要差别：

- XHTML页面和浏览器必须严格地遵守标准

- 所有的标签和属性必须是小写的
- 结束标签是必须的
- 属性的值必须被包含在引号中
- 标签的嵌套必须是正确的
- 每个文档必须指明它的文档类型

HTML

HTML元素/标签

HTML元素：元素 = 开始标签 闭合标签 内容

- 开始标签：包含元素的名称，被开闭尖括号包围，表示元素从此开始或者开始起作用
- 闭合标签：在元素名之前加了一个斜杠，表示元素在此结束
- 内容：包含一个元素的内容

特点：

- 大小写不敏感，推荐小写
- 可嵌套
- 存在自闭合元素，没有结束符号，没办法在内部插入其他标签或文字，只能定义属性

属性：包含了关于元素的额外的信息，这些信息本身并不会被显现在内容中，包含：

- 属性与元素名称（或上一个属性，如果有超过一个属性的话）之间的空格符
- 属性的名称，并接上一个等号
- 由引号所包围的属性值，单双引号均可

布尔属性：没有写属性值的属性，合法的属性值只有空字符串（等价于不写）和与属性名同名的字符串两种，其它都是非法的

块级元素 VS. 内联元素：

- 块级元素（block）：
 - 在页面中以块的形式展现，即相对与其前面的内容它会出现在新的一行，其后的内容也会被挤到下一行呈现。
 - 通常用于展示页面上结构化的内容，例如段落、列表、导航菜单、页脚等等。
 - 注：块级元素不会被嵌套进内联元素中，但可以嵌套在其它块级元素中。
 - 常见：<p>、<h1>-<h6>、<div>、 、<form>
- 内联元素（inline）：
 - 通常出现在块级元素中并包裹文档内容的一小部分，而不是一整个段落或者一组内容。
 - 内联元素又称为行内元素。
 - 不会导致文本换行：它通常出现在一堆文字之间。
 - 常见：、、<a>

文档结构

文档 = 文档声明 元素(e 元素 元素)

文档声明： `<!DOCTYPE>`

- 不是 HTML 标签，它是指示 web 浏览器关于页面使用哪个HTML版本进行编写的指令
- 必须是 HTML 文档的第一行，位于标签之前，没有结束标签，对大小写不敏感
- 最短的有效文档声明： `<!DOCTYPE html>`

`<html>`元素：

- 包裹了整个完整的页面，是一个根元素，所有其他元素必须是此元素的后代。
- `<html>` 与 `</html>` 标签限定了HTML文档的开始点和结束点，在它们之间是文档的头部和主体，文档的头部由 `<head>` 标签定义，而主体由 `<body>` 标签定义。

`<head>`元素：

- 用于定义文档的头部，它是所有头部元素的容器。
- 其中的元素可以引用脚本、指示浏览器在哪里找到样式表、提供元信息、想在搜索结果中出现的关键字和页面描述、字符集声明等。
- 描述了文档的各种属性和信息，包括文档的标题、在 Web 中的位置以及和其他文档的关系等。绝大多数文档头部包含的数据都不会真正作为内容显示给读者。
- `<title>` 元素定义文档的标题，它是 `<head>` 元素中唯一必需的元素。

`<body>`元素：

- 定义文档的主体。
- 包含文档的所有内容，比如文本、超链接、图像、表格和列表等等。

文档头部

`<title>`：添加网页标题

- 定义浏览器工具栏中的标题
- 提供页面被添加到收藏夹时的标题
- 显示在搜索引擎结果中的页面标题

`<meta>`：添加网页元信息（SEO）

- `name + content`：定义作者、关键词、描述、版权等
 - 名称/值：用于SEO（搜索引擎优化）
 - `name`属性提供了名称/值对中的名称（`keywords/description/author/copyright`）
 - `description`给搜索引擎提供关于这个网页的简短的描述
 - `keywords`给搜索引擎提供描述这个网页的关键词（小心使用，3-4个重要且相关关键字即可）
 - `content`属性提供了名称/值对中的值

- robots/robotterms：设置搜索引擎爬虫的交互策略
 - 写法：`<meta name="robots" content="指令1, 指令2, ...">`
 - robotterms=all | none | index | noindex | follow | nofollow
 - index：搜索引擎索引此网页
 - follow：搜索引擎继续通过此网页的链接索引其它网页
 - noindex：搜索引擎不索引此网页
 - nofollow：搜索引擎不继续通过此网页的链接索引其它网页
 - all：等价于index, follow
 - none：等价于noindex, nofollow，搜索引擎将忽略此网页
 - 使用Robots排除协议（Robots Exclusion Protocol）设置搜索引擎爬虫的交互策略 => robots.txt：在网站的根目录下放置robots.txt，说明网站中哪些网页是爬虫不能索引的，文件中User-agent、Disallow这两个部分是必需的。
 - User-agent指所针对的是哪个爬虫。* 表示所有的爬虫，也可以指定一个或若干个爬虫。；
 - Disallow是告诉爬虫哪些路径是不能访问的。
- http-equiv + content：回应给浏览器一些有用信息，帮助正确和精确地显示网页内容，如文本类型（Content-Type）、重定向（Refresh）、过期时间（Expires）等。
- 小结：
 - description被用来显示有关网页内容的信息，搜索引擎会在搜索引擎结果页面（SERP）使用它
 - robots用来阻止搜索引擎获取拷贝页面、私密页面和未完成的页面
 - title最重要，建议控制在70个字符以下，并在其中使用关键词
 - keywords标签时代已经过去，最好不再使用

`<link>`：

- 为站点添加自定义图标，图标出现在浏览器标签页以及书签中
- `<link rel="" href="" type="">`：
 - rel：当前页面和目标资源之间的关系，如stylesheet、icon/favicon等
 - href：目标资源URL
 - type：MIME类型，必要时才写

在HTML中应用CSS和JavaScript:

- CSS：
 - 利用 `<link>` 元素引用外部CSS样式文件：`<link rel="stylesheet" href="xxx.css">`
 - 利用 `<style>` 元素定义CSS样式：`<style type="text/css">...</style>`
- JS：
 - 利用 `<script>` 元素引用外部js脚本文件：`<script src="xxx.js"></script>`
 - 注：放在文档尾部是更好的选择，确保在加载脚本之前浏览器已经解析了HTML内容

- 直接将脚本放入 `<script>` 元素中：`<script type="text/javascript">...</script>`

文档主体

段落与文字

标题： `<h1>` - `<h6>` ，定义多级标题

- 有序的，一般 `<h1>` 只有一个，有助于搜索引擎理解网页
- 强调：确保网页语义结构清晰良好

段落： `<p>` 定义段落，`<br>` 元素在不产生新段落的情况下进行换行。

- 段落标签会自动换行
- 对比：效果类似，但是语义有很大不同：
 - `<br>` 标签是给文字换行，不会带来空隙
 - `<p>` 标签用来给文字分段，段落间有空隙

文本格式化标签：

- `<em>`：斜体，用于标记需要用户着重阅读的内容（不建议 `<i>`）
- `<strong>`：粗体，用于标记非常重要的内容（不建议 `<b>`）
- `<sup>/<sub>`：用于标记上/下标
- `<time>`：将时间和日期标记为可供机器识别的格式
- `<abbr>`：用于标记缩略语并在光标移动到上面时提供解释
- `<address>`：用于标记联系方式
- `<hr/>`：用于标记水平线
- `<del>`：删除线（不建议 `<s>`）
- `<ins>`：下划线（不建议 `<u>`）
- 展示计算机代码：
 - `<code>`：标记计算机通用代码
 - `<pre>`：保留空格（通常是代码块）
 - `<var>`：标记具体变量名
 - `<kbd>`：标记输入电脑的键盘输入
 - `<samp>`：标记计算机程序的输出

空白：

- 在HTML中，无论你用了多少空白（包括空白字符，包括换行），当渲染这些代码的时候，解释器会将连续出现的空白字符减少为一个单独的空格符。使用空白为了增加可读性，确保代码被很好地格式化。
- `<pre>`：用于保留空格，通常是代码块

引用：

- 块级引用 `<blockquote>`：如果一个块级内容（一个段落、多个段落、一个列表等）在其他地方被引用，需要被该标签包裹，并在其 `cite` 属性中用URL指向引用资源的位置，可以添加额外样式。
- 行内引用 `<q>`：插入不需要分段的短引用

`<div>` & `<span>`：

- `<div>`：块级元素
 - 没有特定含义，提供组合其他HTML元素的容器，包裹的元素为可以放到body元素内的任何元素，如段落文字、表格、列表、图像等
 - 常见用途：
 - 与 CSS 一同使用，为大块内容设置样式属性
 - 用于文档布局
- `<span>`：内联元素
 - 没有特定含义，可以用来组合内联元素，以便通过样式来格式化它们
 - 可以为 `span` 设置id或class属性，这样既可以增加适当的语义，又便于对 `span` 应用样式
- 为什么 `<div>` 可以包裹任何元素而 `<span>` 只能包括内联元素？
 - 块级元素在浏览器默认显示状态下将独占整行，排斥其他元素与其位于同一行，块级元素默认为矩形，其中可容纳内联元素或块级元素；内联元素默认显示状态可以与其他内联元素共存于一行，内联元素可以容纳内联元素，但不可容纳块级元素。

注释：`

特殊字符：

<code>&lt;</code>	<code><</code>
<code>&gt;</code>	<code>></code>
<code>&amp;</code>	<code>&</code>
<code>&quot;</code>	<code>"</code>
<code>&apos;</code>	<code>'</code>

列表

有序列表： `<ol>` + `<li>`

- 设置编号属性
 - `start` 定义从哪个数字开始计数
 - `reversed`：启动列表倒计数，布尔属性

无序列表： `<ul>` + `<li>`

描述列表： `<dl>`

- 列表项： `<dt>`
- 描述内容： `<dd>`

列表项符号定义：

- `type` 属性（不推荐）：
 - 有序：通过定义第一个序号（如 `type="1/a/A/i/I"`）定义类型
 - 无序： `type="disc/circle/square"`（实心圆/空心圆/实心正方形）
- CSS的 `list-style-type` 属性定义（推荐）

子元素（`<li>`）

- `<ol>/<ul>` 内部不应存在任何其他标签，一般情况下只能存在 `<li>` 标签
- `<li>` 元素为块级元素，其中可以包含任意元素

表格

基本语义：实现二维表的数据展示

元素定义：

- `<table>`：表格
- `<tr>`：行
- `<td>`：单元格，有 `colspan` 和 `rowspan` 属性，允许单元格跨越多行和多列
- `<caption>`：表格标题，默认位于第一行，一个表格只能有一个
- `<th>`：定义列和行的表头单元格，浏览器默认会以粗体和居中的样式显示，有 `colspan` 和 `rowspan` 属性，允许单元格跨越多行和多列

表格语义化：

- 使用 `<thead>`，`<tfoot>` 和 `<tbody>` 元素，把表格中的部分标记为表头、页脚、正文部分
- 优点：
 - 有助于实现语义清晰的复杂表格结构
 - 方便分块控制表格的CSS样式

多媒体与嵌入

`<img>`：嵌入图片

- 语法： `<img src="" alt="搜索引擎看的描述" title="用户看的描述">`
- `alt` 属性定义备选文本
- 注意：
 - 搜索引擎也读取图像的文件名并把它们计入SEO，因此应该给图片取一个描述性的文件名

- 搜索引擎主要依赖alt属性判断图片含义，所以在图片alt属性中提供包含关键词的简要文字说明也是网页SEO的一部分
- 得到授权前不要将 src 指向其他人网站上的图片（“盗链”）

<figure> & <figcaption>

- 为图片提供语义容器，在标题和图片之间建立关联，实现语义良好 图片 图注 效果
- 语法：

```
<figure>
<img>...
<figcaption> xxx </figcaption>
</figure>
```

- 注：<figure> 里面里不一定要是一张图片，可以是几张图片、一段代码、音视频、方程、表格或别的

图片格式：

- 位图：使用像素点来描述图像，也称为点阵图像
 - JPG：可以很好处理照片等有大面积色调的图像，但保证图片清晰、逼真时文件较大；
 - PNG：体积小，无损压缩，保证网页打开速度，支持透明信息；
 - GIF：图像效果很差，但可以制作动画
- 矢量图：使用线段和曲线描述图像，所以称为矢量，同时图形也包含了色彩和位置信息

对比	矢量图	位图
与分辨率的相关性	无关	有关
色彩丰富度	低，常用于logo、图标	高
常见文件类型	*.AI、*.EPS、*.SVG等	*.bmp、*.pcx、*.gif、*.jpg、*.tif、*.png等
占用空间	小	大

<video>：插入视频

- 插入多个 <source> 标签解决多格式支持问题，浏览器将会检查 <source> 标签，并且播放第一个与其自身解码程序相匹配的媒体
- 视频应当包括 WebM 和 MP4 两种格式，这两种目前大多数平台和浏览器都支持

```
<video controls>
  <source src="xxx.mp4" type="video/mp4">
  <source src="xxx.webm" type="video/webm">
</video>
```

<audio>：插入音频

- 除weight/height/poster等视觉相关属性外，<audio> 支持所有 <video> 拥有的属性

<embed> & <object> 实现通用嵌入

- <embed src="xxx" type="xxx" ...></embed>
- <object data="xxx" type="xxx" width="xxx" height="xxx" ...></object>

<iframe> 嵌入其他HTML网页

- 语法：<iframe src="xxx" width="xxx" height="xxx"></iframe> (src必需)

超链接

<a> 插入超链接：

- 语法：xxx
- 支持HTML中的全局属性（如title），还支持部分个性化属性
 - target 属性控制目标窗口打开方式
 - download 属性提供默认的下载后保存文件名
- 可以链接到HTML文档的指定位置（文档片段），如 名字 指向 id="name" 的元素
- 创建电子邮件链接：href="mailto:xxx@xx.xx?cc=xxx2@xx.xx&subject=xxx&body=xxxxxx"
 - 打开邮件默认客户端自动填写收件人地址
 - cc：抄送
 - subject：自动填写主题
 - body：自动填写正文
 - 包含空格/换行时使用URL编码，确保兼容性
- 创建块级链接：HTML5中，可以通过元素包裹div、h元素和段落标记p等块级元素创建块级链接

href VS. src

- href 是Hypertext Reference的缩写，表示超文本引用，用于在当前文档和引用资源之间建立联系
- src 是source的缩写，指向的内容会嵌入到文档中当前标签所在的位置，用于将指向资源嵌入到当前文档元素定义的位置。当浏览器解析到该元素时，在浏览器下载、编译、执行这个文件之前，页面的加载和处理会被暂停。

表单——<form>

<form>：

- action：规定提交表单时处理表单发送数据的程序的URL
- name：表单名称，用于区分表单
- method：提交表单的HTTP方法，默认GET

- GET：没有敏感信息时使用。通过URL提交数据，可提交的数据量跟URL的长度有直接关系，HTTP协议规范对URL长度没有限制，URL长度受浏览器和Web服务器限制；
- POST：包含敏感信息时使用（不可见），实际开发常用
- target：同 `<a>`
- accept-charset：规定服务器处理表单数据所接受的字符集
- enctype：规定表单向提交服务器的内容的编码方式，默认 `application/x-www-form-urlencoded`，发送到服务器前所有字符都会进行编码
- autocomplete：规定浏览器是否能够根据用户之前输入的值自动补全
- novalidate：规定浏览器是否启用form元素的原生校验机制

表单对象

放在 `<form>` 内的各种标签，也称为表单部件，下面详细介绍：

`<input>`：输入框

- 最基本的表单小部件，提供非常便捷方式让用户输入任何类型的数据
- 通过设置 `type` 属性，它可以彻底变成大多数类型的表单小部件，包括单行文本字段、没有文本输入的控件、时间和日期控件和按钮等
 - text：文本框
 - password：密码文本框
 - radio：单选按钮
 - `<input type="radio" name="单选按钮所在的组名" value="单选按钮的取值"/>`
 - 必须设置 `name` 和 `value` 属性
 - 对于同一个问题的不同选项必须要设置一个相同的`name`属性值，这样才能把这些选项归为同一个问题上
 - checkbox：复选框
 - `<input type="checkbox" name="复选框名称" value="复选框取值" checked="checked"/>`
 - 必须设置 `name` 和 `value` 属性
 - 没有文本，需要加入label标签，并用label标签的`for`属性指向复选框的id（语义化）
 - 按钮：
 - button：普通按钮，`<input type="button" value="显示在按钮上的文字" onclick="JavaScript脚本程序"/>`
 - submit：提交按钮，将所在表单内的所有数据发送到服务器
 - reset：重置按钮，将用户在所在表单输入的信息重置为默认值
 - 注：提交和重置按钮只针对当前所在form标签内的表单元素内容有效，对当前所在form标签之外的表单元素无效
 - image：图片域，`<input type="image" src="xxx">`
- 通用规范：

- 可以被标记为 `readonly`（用户不能修改输入值）甚至是 `disabled`（输入值永远不会与其余表单数据一起发送）
- 可以有一个 `placeholder` 属性，这是输入框中出现的文本，用来简略描述输入框的目的
- 被限制在 `size`（框的物理尺寸）和 `maxlength`（可以输入的最大字符数）之中

`<select>` & `<option>` & `<optgroup>`：下拉列表

- 列表由 `<select>` + `<option>` 实现
- `<optgroup>` 设置选项分组
- 层级为 `<select>` > `<optgroup>` > `<option>`
- `<option>` 设置 `selected` 属性代表默认值
- 可以设置 `multiple` 属性，通过 `Ctrl/Shift+鼠标左键` 实现多选

`<textarea>`：多行输入字段（文本域）

- `<textarea rows="行数" cols="列数">多行文本框内容</textarea>`

`<datalist>` & `<option>`：自动补全输入框

- 提供一个下拉框向用户展示输入可能匹配的内容
- 使用 `<datalist>` 为 `<input>` 提供可能值，由 `<option>` 指定，使用 `list` 属性将数据列表绑定到一个文本域（一般是input），关联后选项内容用于自动完成用户输入脚本

```
<input type="text" list="Suggestion">
<datalist id="Suggestion">
  <option>xxxx</option>
  <option>xxxx</option>
  <option>xxxx</option>
  <option>xxxx</option>
</datalist>
```

`<fieldset>` & `<legend>`：创建控件组

- `<fieldset>` 对表单控件进行分组，一个表单可以有多个控件组
- `<legend>` 用于对每组内容进行描述
- 用于定义语义结构良好的表单

`<label>`：为元素设置标签，用于定义语义结构良好的表单

- `<label for="id">xxx</label>`

文档与网站架构

基本组成部分：

- 页眉：通常横跨于整个页面顶部有一个大标题和/或一个标志，这是网站的主要一般信息，通常存在于所有网页
- 导航栏：指向网站各个主要区段的超链接。通常用菜单按钮、链接或标签页表示
- 主内容：中心的大部分区域是网页的独有内容，如视频、文章、地图、新闻等，这些内容是网站的一部分，会因页面而异
- 侧边栏：一些外围信息、链接、引用、广告等。通常与主内容相关（例如一个新闻页面上，侧边栏可能包含作者信息或相关文章链接），还可能存在其他的重复元素，如辅助导航系统
- 页脚：横跨页面底部的狭长区域，和标题一样，页脚是放置公共信息（如版权声明或联系方式）的，一般使用较小字体，且通常为次要内容

语义：

- 敬畏语义，需要正确选用元素实现网页布局，视觉效果并不是一切
- 语义化标签：
 - `<header>`：页眉
 - `<nav>`：导航栏，包含页面主导航功能，其中不应包含二级链接等内容
 - `<main>`：主内容
 - 存放每个页面独有的内容，每个页面上只能用一次且直接位于 `<body>` 中，最好不要把它嵌套进其它元素
 - 可以包括各种内容区域，如 `<article>`、`<section>`、`<div>` 等
 - `<article>`：一篇文章，与页面其它部分无关
 - `<section>`：更适用于组织页面使其按功能分块
 - `<aside>`：侧边栏，经常嵌套在 `<main>` 中
 - `<footer>`：页脚

信息架构设计：

- 规划整个网站的内容，要可能带给用户最好的体验，需要哪些页面、如何排列组合这些页面、如何互相链接等问题不可忽略
- 规划步骤：
 - 大多数（不是全部）页面会使用一些相同的元素，例如导航菜单以及页脚内容，若网站为商业站点，不妨在所有页面的页脚都加上联系方式，记录这些对所有页面都通用的内容
 - 为页面结构绘制草图，记录每一块的作用
 - 对于期望添加进站点的所有其它（通用内容以外的）内容直接罗列出来
 - 对这些内容进行分组，这样可以让你了解哪些内容可以放在同一个页面
 - 绘制一个站点地图的草图，使用一个气泡代表网站的一个页面，并绘制连线来表示页面间的一般 workflow

HTML验证

通过 MarkupValidation Service，一个由W3C创立并维护的标记验证服务

层叠样式表CSS

概念

基础

CSS (Cascading Style Sheet)：一种样式表语言（HTML是标记语言），用来描述 HTML 或 XML（包括如 SVG、XHTML 之类的 XML 分支语言）文档的呈现。描述了在屏幕、纸质、音频等其它媒体上的元素应该如何被渲染的问题。

CSS2.1是推荐标准，CSS3是新版本（包括border-radius、box-shadow、flexbox等新特性）

工作原理

DOM是一种树形结构，标记语言中的每个元素，属性，文本片段都变为一个 DOM 节点。节点的关系有父子、兄弟等。浏览器渲染引擎会通过计算，将对应的 CSS 声明应用到页面的每一个元素上，从而使得元素们以适当的方式布局，并展示出适当的样式。

引用

外部样式表：

- 在文件中单独定义，并在HTML的 `<head>` 元素中使用 `<link>` 来引用
- 当样式需要被应用到多个页面时，使用外部样式表是理想的选择

内部样式表：

- HTML代码和CSS代码在一个文件中，CSS代码在HTML `<head>` 元素中的 `<style>` `</style>` 标签对内
- 不如外部样式表高效，会增加每个网页的大小和加载时间，让用户等待更长时间
- 在某些特殊情况下使用，不推荐

内联模式：

- HTML代码和CSS代码在一个文件中，CSS代码仅影响一个元素，在元素的 `style` 属性中定义
- 冗余代码多，难以维护
- 在某些特殊情况下使用，不推荐

CSS语法

基础

CSS规则：

- 由选择器和声明块两部分组成
 - 选择器：一种模式，在页面上匹配一些元素，是的相关的声明仅被应用到被选择的元

素上

- 声明块：包括一条或多条声明，每条声明由一个属性和一个值组成，属性（Property）是希望设置的样式属性（style attribute）
- 每个规则（除了选择器的部分）都应该包含在成对的大括号里
- 每个声明里应该用冒号分离属性与属性值
- 在每个规则集里，应该用分号分离各个声明
- 通过添加空格空行使样式表更具可读性
- 注释： `/* xxx */`
- 简写属性：如font, background, padding, border、margin等允许在一行设置多个属性，节省时间并使代码更整洁
- 大小写不敏感，存在例外是选择器中的class和id名称对大小写敏感，推荐全小写

选择器

用于定位想要样式化的HTML元素

分类：

- **简单选择器**：通过元素类型、class 或 id 匹配一个或多个元素
- **属性选择器**：通过属性/属性值匹配一个或多个元素
- **伪类**：匹配处于确定状态的一个或多个元素，例如被鼠标指针悬停的元素，或当前被选中或未选中的复选框，或元素是DOM树中一父节点的第一个子节点
- **伪元素**：匹配处于相关的确定位置的一个或多个元素，例如每个段落的第一个字，或者某个元素之前生成的内容
- **组合选择器**：以有效的方式组合两个或更多的选择器用于实现特定选择的方法，例如，只选择divs的直系子节点的段落，或者直接跟在headings后面的段落
- **多重选择器**：将以逗号分开的多个选择器放在一个CSS规则下面， 以将一组声明应用于由这些选择器选择的所有元素

简单选择器

基于元素的类型（或其 class或 id）直接匹配文档的一个或多个元素，分为元素选择器、标识（id）选择器、类（class）选择器。

- 元素选择器：选中所有元素名和选择器名匹配的元素（不区分大小写）
- id选择器：基于id选择—— `#id` （区分大小写）。元素的id属性：
 - 用于标识网页元素的唯一身份，在同一HTML页面中，id应该是唯一的，在不同页面是可以出现相同id的元素
 - 主要作用是帮助CSS选择器或者JavaScript选中唯一元素
- class选择器：基于class选择—— `.class` （区分大小写）。元素的class属性：
 - 定义元素的类名称。同一元素可以设置多个类名，用空格分隔。同一页面，相同或不同的元素可以具有相同的类名。类名不能以数字开头
 - 可以为同一个页面的相同元素或者不同元素设置相同的class，CSS选择器通过class

属性选择具有同一属性值的多个元素，使其具有相同的CSS样式

属性选择器

一种特殊类型的选择器，它根据元素的属性和属性值来匹配元素，通用语法由 `[]` 组成，其中包含属性名称，后跟可选条件以匹配属性的值。根据其匹配的方式可以分为如下两类：

- 根据属性的存在或属性值进行精确匹配
 - `[attr]`：选择带有以 `attr` 命名的属性的元素
 - `[attr=value]`：选择带有以 `attr` 命名的，且值为"value"的属性的元素
 - `[attr~=value]`：选择带有以 `attr` 命名的属性的元素，并且该属性是一个以空格作为分隔的值列表，其中至少一个值为"value"
- 根据属性值的子串进行类正则表达式的非精确匹配
 - `[attr|=val]`：选择`attr`属性的值是 `val` 或值以 `val-` 开头的元素
 - `[attr^=val]`：选择`attr`属性的值以 `val` 开头（包括 `val`）的元素
 - `[attr$=val]`：选择`attr`属性的值以 `val` 结尾（包括 `val`）的元素
 - `[attr*=val]`：选择`attr`属性的值中 包含子字符串 `val` 的元素

伪类和伪元素选择器

伪类选择器：以冒号为分割添加到选择器末尾，用来选择特定状态下的元素

- 状态伪类（优先级从高到低）：
 - `:link`：表示链接的正常状态，选择那些尚未被点过的链接
 - `:visited`：选择点过的链接
 - `:focus`：用于选择已经通过指针设备、触摸或键盘获得焦点的元素，在表单里常用
 - `:hover`：在用户指针悬停时生效，不只适用于链接
 - `:active`：选择被鼠标指针或触摸操作“激活的”元素，只发生在鼠标被按下到被释放的这段时间里
- 结构伪类：
 - `:first-child`：选择是父元素第一个子元素的指定类型元素
 - `:last-child`：选择是父元素最后一个子元素的指定类型元素
 - `:nth-last-child(i)`：选择是父元素倒数第*i*个子元素的指定类型元素
 - `:first-of-type`：选择是父元素指定类型子元素中第一个元素的元素（同级的相同元素的第一个）
 - `:last-of-type`：选择是父元素指定类型子元素中最后一个元素的元素（同级的相同元素的最后一个）
 - `:nth-of-type(i)`：选择父元素指定类型子元素中的第*i*个子元素（同级的相同元素的第*i*个）
 - `:nth-last-of-type(i)`：选择父元素指定类型子元素中倒数第*i*个子元素（同级的相同元素的倒数第*i*个）
 - `:nth-child(i)`：根据元素在标记中的次序选择相应的元素，先找到第*i*个子元素，然后判定类型是否与冒号前类型匹配，匹配则生效

- `:only-child`：选择是父元素中唯一子元素的所有元素（独生子女）
- `:not`：伪类选择与参数不匹配的所有元素
- `:only-of-type`：选择是父元素的指定类型唯一子元素的所有元素（同级没有同类）
- 验证伪类：
 - `:checked`：选择被勾选或选中的单选按钮、多选按钮及列表选项
 - `:default`：从表单中选择默认的元素
 - `:disabled`：选择禁用状态的表单元素，`:enabled`
 - `:empty`：选择其中不包含任何内容的空元素
 - `:in-range`：选择有范围且值在指定范围内的元素，`:out-of-range`
 - `:indeterminate`：选择页面加载时没有勾选的单选按钮或复选框
 - `:valid`：选择输入格式符合要求的表单元素，`:invalid`
 - `:optional`：选择表单中非必填的输入字段
 - `:read-only`：选择用户不能编辑的元素，`:read-write`、`:required`
- 语言伪类：
 - `:lang()`：基于元素语言来匹配页面元素

伪元素选择器：以双冒号为分割添加到选择器末尾，用来选择元素的某个特定部分

- 伪元素：一种虚拟元素，可以将其视为普通的HTML元素，但是不存在于DOM树中，不能在HTML中输入，只能通过CSS创建伪元素
- 伪元素选择器：
 - `::before` & `::after`：其他HTML元素添加内容（文本或图形），添加的内容不实际存在于DOM中，但可以像存在一样操作它们，需要在CSS中声明content属性，表示在某个元素内容的最前/后面自动插入一段虚拟的内容
 - `::first-letter`：选择一行文本第一个字符，并且文字所处的行之前没有其他内容，只适用于块级元素，内联元素不适用（可以选中 `::before` 伪元素生成的第一个字符）
 - `::first-line`：选择元素的第一行，只适用于块级元素，内联元素不适用
 - `::selection`：选择文档中被高亮选中的部分，基于Gecko的浏览器要使用 `::-moz-selection`

组合选择器和多重选择器

名称	组合	Select
群组选择器 Group of selectors	A, B	匹配满足A和B的任意元素
后代选择器 Descendant selector	A B	匹配任意元素，满足条件：B是A的在DOM树上的后代结点（B是A的子节点，或者A的子节点的子节点）
子元素选择器 Child selectors	A > B	匹配任意元素，满足条件：B是A的直接子节点
相邻兄弟选择器 Adjacent sibling selector	A + B	匹配任意元素，满足条件：B是A的下一个兄弟节点（AB有相同的父结点，并且B紧跟在A的后面）
后继兄弟选择器 General sibling selector	A ~ B	匹配任意元素，满足条件：B是A之后的兄弟节点中的任意一个（AB有相同的父节点，B在A之后，但不一定是紧挨着A）

通配选择器

*：匹配任意类型的HTML元素

样式

注意：

- 在CSS中属性和值都是区分大小写的
- CSS的取值除了关键字和数值之外，取值还可以为函数，如 `width: calc(90% - 30px);`

文本和段落样式

- **文本的展示效果**：用于设置字体、字号、阴影等样式，会直接应用到文本中，比如使用哪种字体、字体的大小是怎样的、字体是粗体还是斜体等
 - `font-family`：字体类型，可指定多种字体，多个字体将按优先顺序排列，以逗号隔开
 - `font-size`：字体大小，值为**关键字/值**
 - 关键字：
 - 绝对大小关键字：xx-small, x-small, small, medium, large, x-large, xx-large
 - 相对大小关键字：larger, smaller（相对父元素）
 - 值：像素px或em数字值或百分比等。px用于精确控制，而em则常用于创建灵活的布局和响应式设计。pt（点）主要用于打印场景，因此在网页设计中使用较少
 - 像素px，浏览器默认字体大小是 16 像素
 - em值，1em等于1倍继承字体大小
 - 百分比，相对于继承字体大小的百分比
 - `font-weight`：字体粗细，值为**关键字/值**
 - 关键字：normal、lighter、bold、bolder
 - 值：100-900，越大越粗
 - normal=400，bold=700（常用）
 - `font-style`：字体样式
 - normal：默认，正常体
 - italic：斜体
 - oblique：让没有倾斜属性的文字倾斜
 - `text-decoration`：文本装饰，可一次性设定多个值。（text-decoration 是缩写，由 text-decoration-line, text-decoration-style 和 text-decoration-color 构成）
 - none：默认，常用于去掉 `<a>` 的默认下划线
 - underline：下划线，重点表明
 - line-through：删除线，常用于促销
 - overline：顶划线，很少见
 - `text-transform`：文本转换

- none
- UPPERCASE：全大写
- lowercase：全小写
- Capitalize：首字母大写
- text-shadow：文本阴影，： 水平偏移 垂直偏移 模糊半径 颜色（偏移值可为负）
 - 可通过包含以逗号分隔的多个阴影值，将多个阴影应用于同一文本
- color：颜色，设置选中元素的前景内容的颜色（通常指文本，也包含文本下方或上方的线），取值可以为：
 - 十六进制颜色： #RRGGBB)
 - RGB颜色： rgb(red, green, blue)
 - RGBA颜色： rgba(red, green, blue, alpha)， alpha 为透明度 (0~1)
 - HSL色彩： HSL(色相, 饱和度, 明度) (0~360, 0~100%, 0~100%)
 - HSLA, 在HSL基础上加透明度
- **文本的布局风格**：用于设置文本的间距以及其他布局功能，比如设置行与字之间的空间，以及在内容框中文本如何对齐等
 - text-indent：文本缩进
 - 属性值：像素、em值（可以是负数）、百分比（相对所在块宽度的百分比）等作为属性值，设置缩进大小
 - 作用：
 - 定义段落的首行缩进
 - 原来插入用HTML空格实现()，现在可以定义样式实现
 - 段落首行缩进的是两个字的间距，如果来实现这个效果，text-indent的属性值应该是字体font-size属性值的两倍
 - text-align：文本对齐
 - 属性值：left（左对齐）、right（右对齐）、center（居中）、justify（使文本展开，改变单词之间的差距，使所有文本行的宽度相同）
 - 作用：
 - 控制文本水平方向的对齐方式
 - 不仅对文本文字有效，对img标签也有效
 - 只能针对文本文字和img标签，对其他标签无效
 - line-height：行高，指的是“一行的高度”，接收<数字>、<长度>、<百分比>、关键字 normal作为属性值，设置行的高度
 - letter-spacing：字母间距
 - normal：默认，无额外空间
 - length：定义字符间固定空间
 - inherit：从父元素继承
 - word-spacing：词间距（同上）

列表样式

- list-style-type：设置列表项符号的类型

- `list-style-position`：设置列表项符号位置，`outside`（默认）/ `inside`
 - `list-style-image`：设置列表项符号图片，`url("xxx")`
- 注：
- 三个属性可以缩写到一个属性 `list-style`，其中属性值可以任意顺序排列，可以设置 1~3 个值，不指定的属性使用默认值（分别为 `disc`, `none`, `outside`）。
 - 浏览器会优先使用 `list-style-image`，若图片无法加载，才会回退到 `list-style-type`。

链接样式

链接状态：

- `Link`：表示链接没有被访问过的状态，是链接默认状态，可以使用 `:link` 伪类来应用样式
- `Visited`：表示链接被访问过的状态，可以使用 `:visited` 伪类来应用样式
- `Hover`：表示当光标刚好停留在链接之上的状态，可以使用 `:hover` 伪类来应用样式
- `Focus`：表示链接被选中的状态（如通过键盘的 Tab 把焦点移动到这个链接的时候），可以使用 `:focus` 伪类来应用样式
- `Active`：表示链接被激活的状态（如被点击），可以使用 `:active` 伪类来应用样式

默认样式：

- 下划线
- 未访问为蓝色，访问过为紫色
- 悬停在链接上时，光标变成小手图标
- 选中时周围会有轮廓
- 激活链接时变成红色（按住鼠标不松）

样式设置要求必须按顺序书写：`nk e e e`

- 因为链接的样式是建立在另一个样式之上的，比如第一个规则的样式会应用到所有后续的样式

样式设置：

- `:hover`：可以定义任何元素在鼠标经过时的样式
- `cursor` 属性：控制当用户将鼠标悬停在某个 HTML 元素上时显示的指针图标，属性值：
 - `default`：默认，箭头
 - `pointer`：常用，小手
 - `text`：文本 I
 - `wait`：加载转圈
- 去除默认下划线：`text-decoration: none;`
- 添加图标：


```
/* 背景路径，不平铺，背景位置水平方向右对齐/垂直方向顶端对齐 */
background: url("xxx") no-repeat 100% 0;
/* 宽度 高度 */
background-size: 16px 16px;
/* 元素增加内边距容纳图标 */
padding-right: 16px;
```

- 利用列表和链接完成导航菜单：
 - `<ul> > <li> > <a>`
 - `li {display: inline;}`

图片样式

`width & height`：定义图片的宽度和高度

`border`：定义图片边框的宽度、样式和颜色

- 宽度：`border-width`：属性值；
- 样式：`border-style`：属性值；（常用：`solid`）
- 颜色：`border-color`：属性值；
- 综合：`border: width style color;`

`text-align`：定义水平对齐方式（在图片的父元素中定义）

`vertical-align`：定义垂直对齐方式，只对内联元素、表格单元格元素生效（属性值从高到低）

- `top`：顶部对齐
- `middle`：中部对齐
- `baseline`：基线对齐
- `bottom`：底部对齐

`float`：定义文字环绕效果（可以应用于所有的元素）

- 属性值：
 - `left`：左浮动
 - `right`：右浮动
 - `none`：默认，不浮动，显示在其文本中应该出现的位置
 - `inherit`：规定从父元素继承

背景样式

背景颜色

- `background-color`：颜色值；

背景图像：

- `background-image: url(xxx);` : 路径
- `background-repeat` : 显示方式, 如纵向平铺、横向平铺
 - `no-repeat` : 不平铺
 - `repeat` : 默认, 水平垂直同时平铺
 - `repeat-x` : 水平平铺
 - `repeat-y` : 垂直平铺
- `background-position` : 定义背景图像在哪个位置
 - 水平+垂直
 - 可以是具体值、百分比、关键字, 前二者可以混用
 - 如果仅规定一个值, 另一个值为 `center` /
- `background-attachment` : 是否随内容滚动
 - `scroll` : 默认, 背景图像对象滚动而滚动
 - `fixed` : 背景图像固定在页面不动

表格样式

`border-collapse` : 表格边框合并

- `separate` : 默认, 边框分开, 不合并
- `collapse` : 边框合并, 若相邻则共用一个边框

`border-spacing` : 指定单元格边界之间的距离。一个属性值同时作用于横向和纵向, 两个属性值分别作用于横向和纵向

层叠和继承

当有多个选择器作用在一个元素上时, 到底哪个规则最终会应用到元素上?

层叠:

- 概念: 当同一个元素被两个选择器选中时, CSS会根据选择器的**权重**决定使用哪一个选择器。权重低的选择器效果会被权重高的选择器效果覆盖。
- 决定因素:
 - CSS规则的重要性和来源
 - CSS规则的特殊性
 - CSS规则在文档中出现的顺序

层叠优先级算法 (步骤):

1. 针对某一元素的某一属性, 列出所有给该属性指定值的CSS规则;
 - CSS规则来源: CSS样式、浏览器默认样式、用户定义的样式
2. 根据声明的重要性和来源进行优先级排序;
 - 重要声明: 利用 `!important` 声明
 - 排序 (从低到高):

- 浏览器默认样式
- 用户样式表的普通声明
- 作者样式表的普通声明
- 作者样式表的重要声明
- 用户样式表的重要声明：如果用户有很好的理由来重写web开发人员样式，可以通过在用户的规则中使用 `!important` 实现

3. 根据选择器的特殊性（Specificity）进行优先级排序

- 选择器特殊性是用四种不同的值来度量的，它们可以被认为是千位、百位、十位和个位：
 - 千位：如果声明是在**style属性**中该列加1分，否则为0（这样的声明没有选择器，特殊性总是1000）
 - 百位：在选择器中每包含一个**ID选择器**就在该列中加1分
 - 十位：在选择器中每包含一个**类、属性、伪类选择器**就在该列中加1分
 - 个位：在选择器中每包含一个**元素选择或伪元素选择器**就在该列中加1分
- 注：通用选择器（*），组合选择器（+, >, ~, ' '）和否定伪类（:not）对特殊性无影响

4. 根据CSS规则在样式文档中出现的位置（Source Order）进行优先级排序，即**后面的规则将战胜先前的规则**

注意：所有这些都发生在属性级别上——属性覆盖其他属性

继承：

- 继承性是指被包在内部的标签默认拥有外部标签的样式性，即子元素可以继承父元素的属性
- 作用：避免同样的内容重复声明，减少CSS文件大小，提升网页加载速度
- 每个属性定义中都指出了这个属性是否默认继承：**通常文本相关属性会默认继承，布局相关属性不默认继承**
- 如果一个元素有多个祖先可以继承相同的属性，但属性值有所冲突，则在DOM树中离子孙最近的那个祖先的属性值会被继承下来

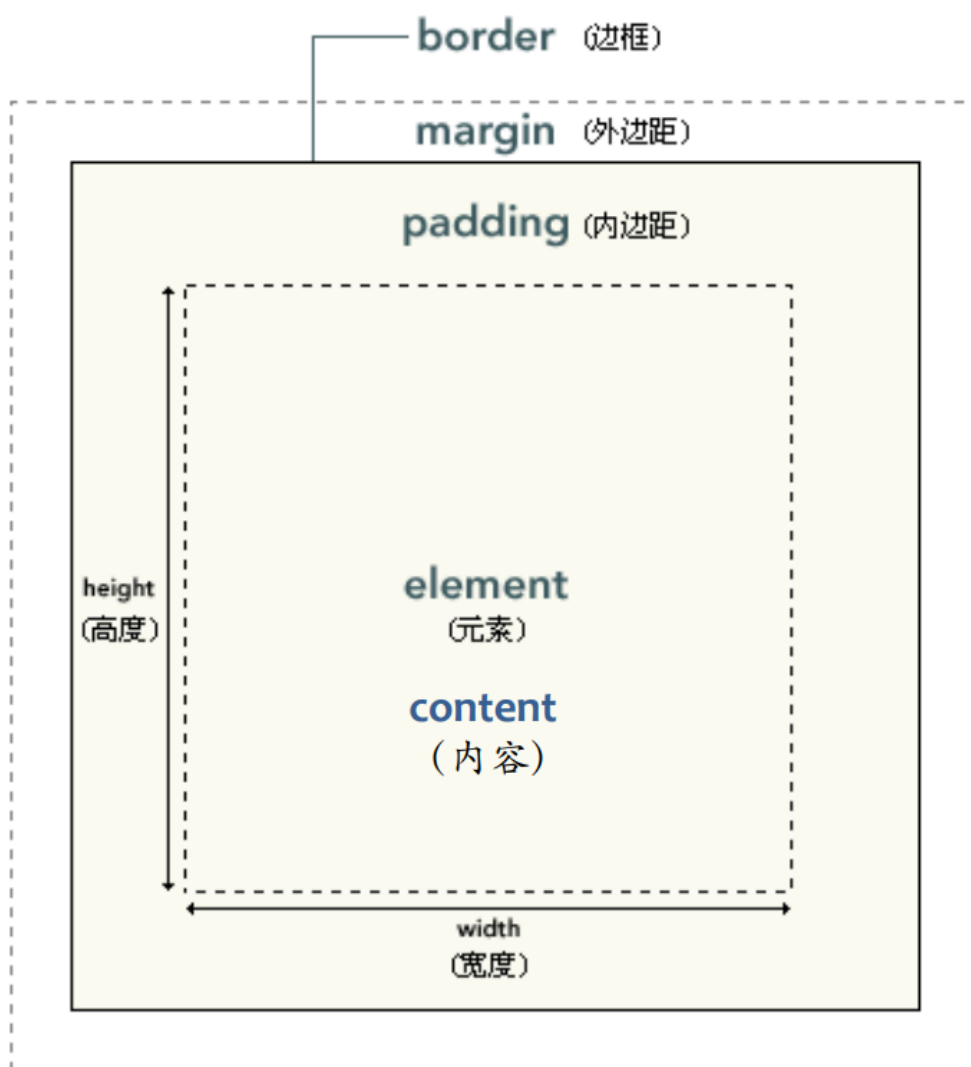
`all` 属性：用于重置或继承元素的所有属性。

- 常见取值：
 - 没有 `all` 属性：按照定义的样式规则渲染。
 - `all: inherit;`：所有样式属性将从父元素继承。如果父元素没有指定这些属性，浏览器将使用默认样式。这种情况下，元素的背景颜色和文本颜色将从父元素获取。
 - `all: initial;`：所有样式属性将重置为其初始值，即浏览器的默认样式。这个选项将忽略自定义样式，而回到浏览器的默认呈现。
 - `all: unset;`：此值将属性设置为 `inherit` 或 `initial`，取决于属性是否继承。非继承属性（如 `background-color`）将重置为初始值，继承属性（如 `color`）则会继承自父元素。
- 注意：IE 和 Safari 不支持 `all` 属性。

样式覆盖常见的冲突方式：

- 引用方式冲突：
 - 行内样式 (内部样式 = 外部样式)
 - 内部样式和外部样式**根据特殊性和源代码顺序进行比较**
- 继承方式冲突：DOM树上最近的祖先获胜
- 指定样式冲突：根据特殊性比较
- 继承样式和指定样式冲突：指定样式获胜
- !important (写在属性值后面)：
 - 如果一个样式用 !important 来声明，则这个样式会覆盖CSS中其他任何非重要样式声明
 - 多个 !important，根据特殊性和源代码顺序进行比较

盒模型



盒模型 (Box Model) 概念

- 网页布局的基础，每个元素被表示为一个矩形的方框，通过设置内容区域、内边距、边框和外边距，可用于浏览器渲染网页布局时计算盒子大小、相对位置等。
- 所有页面中的元素都可以看成一个盒子，并且占据着一定的页面空间。
- 一个页面由很多这样的盒子组成，这些盒子之间会互相影响，因此掌握盒子模型需要从两个方面来理解：

- 一是理解单独一个盒子的内部结构
- 二是理解多个盒子之间的相互关系

术语表：

- 盒模型 box model
- 元素 element
- 内容 content
- 内边距 padding
- 外边距 margin
- 边框 border

元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距。

HTML元素：

1. block：大块内容，有height和width，可以定义四个方向的margin（e.g. `<p>` `<h1>` `<blockquote>` `<ol>` `<ul>` `<table>`）
2. inline：少量内容，没有height和width，只可以定义左右两个方向的margin（e.g. `<a>` `<em>` `<strong>`）
 - 特殊：inline block，有宽高的内联元素，`<img>` `<input>`
3. metadata：页面信息，通常不可见，e.g. `<title>` `<meta>`

block VS. inline

- 块级元素
 - 默认从上到下排列，占据整行，可以设置宽和高
- 内联元素
 - 默认从左向右排列，占用需要的位置，设置宽和高不生效，无法设定位置
 - 想设定位置，需要通过包裹它的元素设置，或者将其 `display` 属性设定为 `block`，占据整行的宽度，并可以设置宽度和高度（如 `text-align`）
- inline block：`<img>` 或者CSS `display`属性设置成 `inline-block` 的任意元素

内容区

- 内容区是CSS盒子模型的中心，它呈现了盒子的主要信息内容，这些内容可以是文本、图片等多种类型
- 内容区是盒子模型**必备的组成部分**，其他的三部分都是可选的
- 属性：`width / height / overflow`
 - 宽度高度只针对内容区，不包括padding部分
 - 只有块级元素可以设置宽高，内联元素不行
 - `overflow`：当内容信息太多时，超出内容区所占范围时，可以使用 `overflow` 溢出属性来指定处理方法

- `overflow`：定义溢出元素内容区的内容会如何处理
 - 属性值：
 - `visible`：默认值，内容不会被修剪，呈现在元素框之外
 - `hidden`：内容会被修剪，且其余内容不可见
 - `scroll`：内容会被修剪，浏览器会显示滚动条，以便查看其余内容
 - `auto`：由浏览器定夺，如果内容被修剪，就会显示滚动条

内边距

- 内容区和边框之间的空间，可以被看做是内容区的背景区域，也常被称为“补白”
- 属性值：
 - 可以通过简写属性 `padding` 一次设置所有边（顺序：`top - right - bottom - left`）
 - 也通过 `padding-top`、`padding-right`、`padding-bottom` 和 `padding-left` 属性一次设置一边

边框

- 默认值为0，可以设置宽度、风格（`none/dotted/solid/dashed/double` 等）、颜色
 - 通过`border`简写属性可以一次设置所有四个边
 - 通过`border-top`, `border-right`, `border-bottom`, `border-left` 分别设置某边的厚度、风格和颜色
 - 通过`border-width`, `border-style`, `border-color`单独设置厚度、风格和颜色并应用到全部四边
 - 可单独设置某边的三个不同属性，如 `border-top-width`, `border-topstyle`, `border-top-color`
- 注：
 - `border-width` 单独使用不起作用，需要先使用 `border-style` 设置边框
 - 默认的边框颜色是元素本身的前景色。如果没有为边框声明颜色，它将与元素的文本颜色相同。如果元素没有任何文本，假设它是一个表格，其中只包含图像，那么该表的边框颜色就是其父元素的文本颜色（因为 `color` 可以继承）
- 使用技巧：
 - 同一元素的边框相交处是斜线，可用来实现三角形（宽高为0，边框设置很粗，三个边框透明即可）
 - `border-radius` 可以为边框设置圆角（IE8-不支持），四值顺序是左上、右上、右下、左下

outline（轮廓）

- 绘制于元素周围的一条线，位于边框边缘的外围，可起到突出元素的作用，可以通过 `outline-color`, `outline-style`, `outline-width`设置样式。
- 设置div宽高都是100px，加了border以后变成102，outline还是100
- border更加灵活，可以指定边框的宽度、类型、颜色和圆角效果；而outline则更适合用于表示元素的状态，例如：`hover`、`focus`或`active`等

背景裁剪 (background clip)

- 背景色、背景图片默认应用于由内容和内边距、边框组成的区域，可以通过设置 background-clip 属性值来调整
 - border-box：默认到边框
 - padding-box：填充到内边距
 - content-box：填充到内容区域

外边距

- 默认值为0，在CSS布局中推开其它盒子
- 可以通过margin或 margin-top、margin-right、margin-bottom 和 margin-left一次性或分布设置宽度、风格和颜色
 - 外边距默认是透明的，不会遮挡其后的任何元素
- 外边距重叠 (collapsing margins)：**相邻+块级元素**的上外边距和下外边距有时会合并为一个外边距，其大小取其中的最大者，这种行为称为外边距折叠
 - 在参与折叠的margin都是正数的情况下，取其中较大的值为最终 margin 值
 - 在参与折叠的margin都是负值的时候，取其中绝对值较大的，然后从零开始负向位移
 - 在参与折叠的margin有正值有负值的时候，要从所有负值中选出绝对值最大的，所有正值中选择绝对值最大的，二者相加
- 自动外边距 (auto margins)：对于块级元素，如果将其margin-left和margin-right设定为 auto，则可将该块级元素居中
- <body> 元素有缺省的外边距 (8px)

内联元素

- 设置宽度和高度不生效，外边距、内边距和边框生效
- 不会改变其他内容与内联盒子的关系，因此内边距和边框会与段落中的其他单词重叠
- 只能设置左右外边距，行间距离需要使用 line-height 控制

内联块元素 (display: inline-block;)

- 设置width和height属性会生效
- padding, margin, 以及border会推开其他元素
- 不会跳转到新行，如果显式添加width和height属性，它只会变得比其实际内容更大

布局

布局过程：选择网页中的元素，并且控制它们相对正常文档流、周边元素、父容器或主视口/窗口的位置。在CSS布局中，可以通过设置某些属性实现“脱离正常文档流”，以控制页面布局。

正常文档流

- 文档流是元素在页面出现的先后顺序，如果没用任何CSS来改变页面布局，HTML元素就会排列在正常文档流（Normal Flow）之中。
- 在正常文档流之中，元素盒子（任何一个HTML元素其实就是一个盒子）会基于文档的书写顺序一个接一个地排列，将窗体自上而下分成一行一行，块元素独占一行，相邻行内元素在每行中按从左到右地依次排列元素
- 确保书写的页面具有良好的语义结构，最大程度复用正常文档流所带来的优势：
 - 正常文档流避免指定所有的HTML元素的布局方式，即使CSS加载失败了，用户仍然能阅读网页内容
 - 确保不使用CSS的工具（例如屏幕阅读器）会按照元素在文档中的位置来读取页面内容
 - 如果网页内容顺序和用户预期的阅读顺序一致，那就不需要为了将元素调整到正确的位置而做大量的布局调整

脱离正常文档流

- 用CSS控制元素的显示位置和文档代码顺序不一致
- `float / position / display`

浮动 `float`

- 允许元素浮动到另外一个元素的左侧或右侧，而不是默认的一个堆叠另一个，导致宽度不再延伸，其宽度为容纳内容的最小宽度
- 主要用途是**布置出多个列并且浮动文字以环绕图片（img加float）**
- 属性值：
 - `left`：将元素浮动到左侧
 - `right`：将元素浮动到右侧
 - `none`：默认值, 不浮动
 - `inherit`：继承父元素的浮动属性
- 如果box1和box2同时为浮动元素，外边距就会生效，如果只有一个是浮动元素则外边距不生效，变成紧贴
- 清除浮动：
 - 一旦对一个元素进行了浮动，所有接下来的元素都会环绕它。
 - 如果不想要在某个元素受到其之前的浮动元素影响时，可以为其添加`clear`属性清除浮动，接下来的部分会以正常流方式绘制在其后
 - 属性值：`left / right / both`

定位 `position`

- 允许我们将一个元素从它在页面的原始位置准确地移动到另外一个位置
- 使用：`position: xxx;`（子绝父相）
- `static positioning`：
 - 静态定位，每个元素默认的属性，表示将元素放在文档布局流的默认位置
 - 静态定位的元素按照文档流正常布局，并不受到其他定位方式的影响

- `relative positioning` :
 - 相对定位，允许我们按照元素在正常文档流中的相对位置移动它
 - 采用相对定位的元素，其位置是相对于它的**原始位置**计算而来的，通过将元素从原来的位置向上、向下、向左或者向右移动来定位的
 - 配合 `top / bottom / left / right` 使用，对应元素相对原始位置四个方向的位移
 - 页面上的其他元素并不会因相对定位元素的位置变化而受到影响，该元素在正常文档流中的位置会被保留
- `absolute positioning` :
 - 绝对定位，将元素完全从页面的正常布局流中移出，将它单独放在一个图层中，设定相对于另一个元素的位置
 - 配合 `top / bottom / left / right` 使用，对应元素相对浏览器四个边的位置
 - 绝对定位的元素的位置相对于最近的已定位父元素，如果元素没有已定位的父元素，那么它的位置相对于 `<html>`
 - 一个元素变成了绝对定位元素，这个元素就**完全脱离正常文档流**，绝对定位元素的前面或者后面的元素会认为这个元素并不存在，即这个元素浮于其他元素上面，它是独立出来的，其原本占据的空间也会被移除
- `fixed positioning` :
 - 固定定位，将元素相对浏览器视口固定，而不是相对另外一个元素
 - 被固定的元素不会随着滚动条的拖动而改变位置，在视野中，固定定位的元素的位置是不会改变的
 - 配合 `top / bottom / left / right` 使用，对应元素相对浏览器四个边的位置
 - 用途很多，一般用于“回顶部”特效和固定栏目的设置

display 属性

值	描述
none	此元素不会被显示。
block	此元素将显示为块级元素，此元素前后会带有换行符。
inline	默认。此元素会被显示为内联元素，元素前后没有换行符。
inline-block	行内块元素。（CSS2.1 新增的值）
list-item	此元素会作为列表显示。
run-in	此元素会根据上下文作为块级元素或内联元素显示。
compact	CSS 中有值 compact，不过由于缺乏广泛支持，已经从 CSS2.1 中删除。
marker	CSS 中有值 marker，不过由于缺乏广泛支持，已经从 CSS2.1 中删除。
table	此元素会作为块级表格来显示（类似 <table>），表格前后带有换行符。
inline-table	此元素会作为内联表格来显示（类似 <table>），表格前后没有换行符。
table-row-group	此元素会作为一个或多个行的分组来显示（类似 <tbody>）。
table-header-group	此元素会作为一个或多个行的分组来显示（类似 <thead>）。
table-footer-group	此元素会作为一个或多个行的分组来显示（类似 <tfoot>）。
table-row	此元素会作为一个表格行显示（类似 <tr>）。
table-column-group	此元素会作为一个或多个列的分组来显示（类似 <colgroup>）。
table-column	此元素会作为一个单元格列显示（类似 <col>）
table-cell	此元素会作为一个表格单元格显示（类似 <td> 和 <th>）
table-caption	此元素会作为一个表格标题显示（类似 <caption>）
inherit	规定应该从父元素继承 display 属性的值。

- 设定display属性值是在CSS中实现页面布局的一种主要方法，此属性允许更改默认显示方式
- display:none：用来隐藏元素，搜索引擎会忽视对应的内容（SEO技巧）
 - 示例：下拉菜单

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
    div.menu-bar ul ul {
        display: none;
    }

    div.menu-bar li:hover {
        color: red;
    }

    div.menu-bar li:hover > ul {
        display: block;
    }
</style>
```

```

</head>
<body>
<div class="menu-bar">
  <ul>
    <li>
      <a href="#">menu</a>
      <ul>
        <li>
          <a href="#">link</a>
        </li>
        <li>
          <a href="#">submenu</a>
          <ul>
            <li>
              <a href="#">submenu</a>
              <ul>
                <li><a href="#">ex1</a></li>
                <li><a href="#">ex2</a></li>
                <li><a href="#">ex3</a></li>
                <li><a href="#">ex4</a></li>
              </ul>
            </li>
            <li><a href="#">link</a></li>
          </ul>
        </li>
      </ul>
    </li>
  </ul>
</div>
</body>
</html>

```

- 自动平均划分元素（把一个块级元素的显示方式改成了表格单元格模式）：
 - 父元素： `display: table;`
 - 子元素： `display: table-cell;`
- 去除 `inline-block` 元素间距：父元素设置 `font-size: 0;`
- 实现 Flexbox 布局：
 - Flexbox是CSS弹性盒子布局模块（Flexible Box Layout Module）的缩写，它被专门设计出来用于创建横向或是纵向的一维页面布局
 - 使用：
 - 父元素（flex容器）： `display: flex;`（所有直接子元素都将会按照flex进行布局）
 - 子元素： `flex: 1;`（每个子元素平分父容器的剩余空间）
 - 除了可以应用到flex容器的属性以外，还有很多属性可以应用到flex项(flex items)上面，这些属性可以改变flex项在flex布局中占用宽/高的方式，允许它们通过伸缩来适应可用空间

- 实现 Grid 布局：
 - Grid布局被设计用于同时在两个维度上把元素按行和列排列整齐
 - 使用：可以通过指定 `display:grid` 来实现grid布局，还可以使用 `grid-template-rows`（行的数量和高度）和 `grid-template-columns`（使用 `fr` 定义份数）两个属性定义行和列

JavaScript

JS导论

介绍

JavaScript是一种轻量级的脚本语言（script language），也是一种嵌入式（embedded）语言

- 脚本语言：它不具备开发操作系统的能力，而是只用来编写控制其他大型应用程序（比如浏览器）的“脚本”
- 本身提供的核心语法不算很多，只能用来做一些数学和逻辑运算，JavaScript 本身不提供任何I/O相关的 API，要靠宿主环境（host，如浏览器、node项目）提供，JavaScript 只合适嵌入更大型的应用程序环境，去调用宿主环境提供的底层 API

JavaScript 的核心语法部分相当精简，只包括两个部分：**基本的语法构造**（比如操作符、控制结构、语句）和**标准库**（就是一系列具有各种功能的对象比如Array、Date、Math等）

各种宿主环境提供额外的API（即只能在该环境使用的接口），以便 JavaScript 调用：

- 宿主环境为浏览器，提供的API分成三大类：
 - 浏览器控制类：操作浏览器
 - DOM类：操作网页的各种元素
 - Web类：实现互联网的各种功能
- 宿主环境为服务器，则会提供各种操作系统的API，比如文件操作API、网络通信API等等。这些 都可以在 Node 环境中找到

优势

- 操作浏览器的能力
 - 发明目的就是作为浏览器的内置脚本语言，为网页开发者提供操控浏览器的能力
 - 目前通用的浏览器脚本语言，所有浏览器都支持。它可以让网页呈现各种特殊效果，为用户提供良好的互动体验
- 广泛的适用领域：JavaScript的使用范围慢慢超越了浏览器，正在向通用的系统语言发展
 - **浏览器的平台化**：随着 HTML5 的出现，浏览器本身的功能越来越强，不再仅仅能浏览网页，而是越来越像一个平台，JavaScript 因此得以调用许多系统功能，比如操作本地文件、操作图片、调用摄像头和麦克风等等。
 - **Node**：Node 项目使得 JavaScript 可以用于开发服务器端的大型项目，网站的前后端都用 JavaScript 开发已经成为了现实。甚至可以应用到嵌入式平台（Raspberry Pi）之上

- **数据库操作：**JavaScript 甚至也可以用来操作数据库。NoSQL 数据库这个概念，本身就是在 JSON（JavaScript Object Notation）格式的基础上诞生的，大部分 NoSQL 数据库允许 JavaScript 直接操作。基于 SQL 语言的开源数据库 PostgreSQL 支持 JavaScript 作为操作语言，可以部分取代 SQL 查询语言
- **移动平台开发：**JavaScript 也正在成为手机应用的开发语言
- **内嵌脚本语言**
- **跨平台的桌面应用程序：**不依赖浏览器
- **易学性**
 - **学习环境无处不在：**几乎所有电脑都原生提供 JavaScript 学习环境，不用另行安装复杂的IDE和编译器（浏览器即可运行）
 - **简单性及主流语言相似性：**语法简单，类似 C/C++ 和 Java
 - **复杂性：**设计大量外部API、存在设计缺陷
- **强大的性能**
 - 语法灵活，表达力强
 - 支持编译运行
 - **WebAssembly：**一种主流浏览器都已经支持的新的字节码格式，它是 JavaScript 引擎的中间码格式，全部都是二进制代码。由于跳过了编译步骤，可以达到接近原生二进制代码的运行速度
 - 事件驱动和非阻塞式设计：可以采用事件驱动（event-driven）和非阻塞式（non-blocking）设计，在服务器端适合高并发环境，普通硬件就可以承受很大的访问量
- 开放性
- 社区支持和就业机会

与其他名词的关系

与Java的关系：

- JavaScript 和 Java 是两种不一样的语言，JavaScript 的基本语法和对象体系，是模仿 Java 而设计的
- 两点最大的区别：
 - 函数是一种独立的数据类型：函数是“第一等公民”（first class），函数与其他数据类型一样，处于平等地位，可以赋值给其他变量，也可以作为参数，传入另一个函数，或者作为别的函数的返回值
 - 采用基于原型对象（prototype）的继承链：每个对象拥有一个原型对象，对象以其原型为模板、从原型继承方法和属性。原型对象也可能拥有原型，并从中继承方法和属性，一层一层、以此类推。这种关系常被称为原型链 (prototype chain)，它解释了为何一个对象会拥有定义在其他对象中的属性和方法

ECMAScript 和 JavaScript 的关系是，前者是后者的规格，后者是前者的一种实现

JavaScript

基础原理

在HTML中的引用方式：

- `<head></head>` 标签内的 `<script type="text/javascript"> ... </script>` 中编写
- `<body></body>` 标签内的 `<script type="text/javascript"> ... </script>` 中编写
- 在元素事件中引入，在元素的某一个属性中（如 `onclick`）直接编写JavaScript程序或调用JavaScript函数，这个属性指的是元素的事件属性
- 引入外部文件，把JavaScript程序存放在一个后缀名为 `.js` 的文件中，然后使用 `script` 标签的 `src` 来引用。`script`标签可以放在`head`标签内，也可以放在`body`标签中
 - 如果脚本文件使用了非英语字符，应该注明字符的编码（`charset="xxx"`），所加载的脚本必须是纯 JavaScript 代码，不能有HTML代码和`script`标签
- `<script>` 标签的 `integrity` 属性：写入该外部脚本的Hash签名，用来验证脚本的一致性

浏览器加载JS的过程

- 主要通过 `<script>` 元素完成，网页加载流程：
 - 浏览器一边下载HTML网页，一边开始解析，不等下载到完，就开始解析
 - 解析过程中，浏览器发现 `<script>` 元素就暂停解析，把网页渲染的控制权转交给JavaScript引擎
 - 如果 `<script>` 引用了外部脚本，就下载该脚本再执行，否则就直接执行代码
 - JavaScript引擎执行完毕后，控制权交还渲染引擎，恢复往下解析HTML网页
- 为什么必须把控制权交给JS而停止页面渲染？
 - JavaScript代码可以修改DOM，所以必须把控制权让给它，否则会导致复杂的线程竞赛的问题
- **阻塞效应**：如果外部脚本加载时间很长（一直无法完成下载），那么浏览器就会一直等待脚本下载完成，造成网页长时间失去响应，浏览器就会呈现“假死”状态
- 将 `<script>` 标签放到页面底部好处：
 - 避免阻塞效应
 - 因为在DOM结构生成之前就调用DOM节点，JavaScript会报错，如果脚本都在网页尾部加载，就不存在这个问题，因为这时DOM肯定已经生成了
- 如果某些脚本代码非常重要，一定要放在页面头部的话，最好直接将代码写入页面，而不是连接外部脚本文件，这样能缩短加载时间
- 脚本的执行顺序由它们在页面中的出现顺序决定，这是为了保证脚本之间的依赖关系不受到破坏，但是下载过程是多个脚本并行的

JavaScript引擎：

- 主要作用：读取网页中的JavaScript代码，对其处理后运行
- JavaScript是一种解释型语言，也就是说，它不需要编译，由解释器实时运行。这样的好处是运行和修改都比较方便，刷新页面就可以重新解释；缺点是每次运行都要调用解释器，系统开销较大，运行速度慢于编译型语言
- 为了提高运行速度，目前的浏览器都将JavaScript进行一定程度的编译，生成类似字节码（bytecode）的中间代码，以提高运行速度

- 早期：代码 -> 字节码 -> 机器码
- 现代：即时编译（Just In Time compiler, JIT），源代码会首先被解释器解释执行，而当某些部分的代码被频繁执行时，JIT编译器会将这些部分的代码编译成机器码，从而提高执行速度（JIT是编译器和解释器的融合/组合）
- 字节码不能直接运行，而是运行在一个虚拟机（Virtual Machine）之上，一般也把虚拟机称为 JavaScript 引擎。并非所有的 JavaScript 虚拟机运行时都有字节码，有的 JavaScript 虚拟机基于源码，即只要有可能，就通过 JIT 编译器直接把源码编译成机器码运行，省略字节码步骤。这一点与其他采用虚拟机（比如 Java）的语言不尽相同。

JavaScript试验环境：浏览器开发者工具控制台

基本语法

语句和表达式

- 执行单位为行
- 语句：为了完成任务而进行的操作，以分号结尾，一个分号就表示一个语句结束，多个语句可以写在一行内，一般情况没有返回值
- 表达式：为了得到返回值的计算式，总是有返回值
- 注：分号前面可以没有任何内容，JavaScript引擎将其视为空语句

变量

- 对“值”的具名引用。变量就是为“值”起名，然后引用这个名字，就等同于引用这个值。
- 变量的名字就是变量名
- JavaScript 的变量名区分大小写
- 如果只是声明变量而没有赋值，则该变量的值是 `undefined`，一个表示“无定义”的特殊值。如果一个变量没有声明就直接使用会报错：变量未定义
- JavaScript 是一种动态类型语言，变量的类型没有限制，变量可以随时更改类型
- 作用域：全局变量（函数外） VS. 局部变量（函数内）
- 数据类型：**原始值类型** 和 **对象（Object）**
 - 前者值就保存在变量指向的那个内存地址，因此 `const` 声明的原始值类型变量等同于常量
 - 后者指向的内存地址其实是保存了一个指向实际数据的指针，所以 `const` 只能保证指针是不可修改的，至于指针指向的数据结构是无法保证其不能被修改的（在堆中）
- `var`、`let`、`const`：声明变量的关键字，后二者为ES6新增
 - `var` VS. `let`：作用域不同
 - `var` 声明的变量的作用域是当前**执行上下文**，如果是在任何函数外面，则是全局执行上下文，如果在函数里面，则是当前函数执行上下文。`var` 声明的变量的作用域只能是全局或者整个函数块的，单独块内重新声明无效，依旧作用到块外原变量上
 - `let` 声明的变量的作用域则是它**当前所处代码块**，即它的作用域既可以是全局或者整个函数块，也可以是 `if`、`while`、`switch`等用大括号限定的代码区块

- `var` 和 `let` 的作用域规则都是一样的，其声明的变量只在其声明的块或子块中可用
- `var` 允许在同一作用域中重复声明，而 `let` 不允许在同一作用域中重复声明，否则抛出异常
- `let` VS. `const`：`const` 与 `let` 很类似，都具有上面提到的 `let` 的特性，唯一区别就在于 `const` 声明的是一个只读变量，声明之后不允许改变其值。因此，`const` 一旦声明必须初始化，否则会报错
 - 本质解释：`const` 保证变量指向的内存地址所保存的数据不允许改动
- 变量提升：JavaScript 引擎的工作方式是，先解析代码，获取所有被声明的变量，然后再一行一行地运行，结果是所有的变量声明语句都会被提升到代码的头部

区块和作用域

- 区块：使用大括号将多个相关的语句组合在一起，称为“区块”(block)，单独使用区块并不常见，区块往往用来构成其他更复杂的语法结构，比如 `for`、`if`、`while`、`function` 等
 - 对于 `var` 命令来说，JavaScript 的区块不构成单独的作用域 (scope)，即在区块内部使用 `var` 命令声明并赋值了变量 `a`，在区块外部变量 `a` 依然有效，区块对于 `var` 命令不构成单独的作用域，与不使用区块的情况没有任何区别
- 作用域：变量在程序中的有效范围（分为全局变量和局部变量）

标识符 (identifier)

- 指的是用来识别各种值的合法名称，常见的标识符有变量名、函数名等，大小写敏感
- 命名规则（不符合则为非法标识符，报错）：
 - 第一个字符可以是任意 Unicode 字母（包括英文字母和其他语言的字母，包括中文）以及美元符号 (\$) 和下划线 (_)
 - 第二个字符及后面的字符，除了 Unicode 字母、美元符号和下划线，还可以用数字 0-9

注释

- 单行：`//`
- 多行：`/* */`
- `<!-- -->` 内若浏览器支持 JS 则解析，否则被当成 HTML 注释

条件语句

- `if(xxx) {...}`
 - `if` 后面的表达式之中，可以有严格相等运算符 (`===`) 和相等运算符 (`==`)，它们的区别是相等运算符 (`==`) 比较两个值是否相等，严格相等运算符 (`===`) 比较它们是否为“同一个值”。如果两个值不是同一类型，严格相等运算符 (`===`) 直接返回 `false`，而相等运算符 (`==`) 会将它们转换成同一个类型，再用严格相等运算符进行比较
- `if(xxx) {...} else if(xxx) {...} ... else {xxx}`

- else代码块总是与离自己最近的那个if语句配对
- switch：

```
switch(xxx) {
  case xxx:
    ...
    break;
  case xxx:
    ...
    break;
  ...
  default:
    ...
}
```

- 注：switch语句后面的表达式，与case语句后面的表示式比较运行结果时，采用的是严格相等运算符，而不是相等运算符，这意味着比较时不会发生类型转换
- 三元运算符（条件）？ 表达式1 ： 表达式2：用于逻辑判断，如果“条件”为true，则返回“表达式1”的值，否则返回“表达式2”的值

循环语句

- while
- do ... while()；：while后面分号不可省略
- for：for(初始化；条件；递增表达式)
- continue & break
- 标签（label）：
 - JavaScript 语言允许，语句的前面有标签，相当于定位符，用于跳转到程序的任意位置
 - 标签可以是任意的标识符，但不能是保留字，语句部分可以是任意语句
 - 标签通常与break语句和continue语句配合使用，跳出特定的循环或者跳出代码块
 - 举例：

```
top:
  for (var i = 0; i < 3; i++) {
    for (var j = 0; j < 3; j++) {
      if (i === 1 && j === 1) break top;
      console.log("i = " + i + "j = " + j);
    }
  }
```

数据类型

JavaScript 语言的每一个值，都属于某一种数据类型。共分为6种：

- 数值（number）：整数和小数
- 字符串（string）：文本

- 布尔值 (boolean)：表示真伪的两个特殊值，true/false
- undefined：表示“未定义”或不存在，即由于目前没有定义，所以此处暂时没有任何值
- null：表示空值，即此处的值为空
- 对象 (object)：各种值组成的集合
 - 分成三个子类型：狭义的对象、数组、函数

分类

- 原子类型：数值、字符串、布尔值
- 复合类型：对象
- 特殊值：null、undefined

typeof 运算符

- 返回一个值的数据类型
- 可能的返回值：number / string / boolean / function / undefined / object
- 注意：null 返回 object

数据类型转换

JavaScript 是一种动态类型语言，变量没有类型限制，可以随时赋予任意值，但是各种运算符对数据类型是有要求的，如果运算符发现运算子的类型与预期不符，就会自动转换类型。

强制类型转换

- 使用 Number() 将各种类型的值强制转换成数字
 - Number函数将字符串转为数值，要比parseInt函数严格，parseInt逐个解析字符，而Number函数整体转换字符串的类型，只要有一个字符无法转成数值，整个字符串就会被转为NaN
 - 参数是原始类型的值：
 - 数值：不变
 - 字符串：可以则转换，不可以返回 NaN
 - 空字符串：转为0
 - 布尔值：true 1 false 0
 - undefined：NaN
 - null：0
 - 参数是对象：
 - 返回 NaN，除非是包含0个或者单个数值的数组
 - 单个数值的数组：转为该数值
 - 0个数值的数组：0
- 使用 String() 将任意类型的值强制转换成字符串：
 - 参数是原始类型：
 - 数值：转为相应的字符串

- 字符串：不变
- 布尔值：true转为字符串"true"，false转为字符串"false"
- undefined：转为字符串"undefined"
- null：转为字符串"null"
- 参数是对象：
 - 狭义对象：一个类型字符串
 - 函数：返回函数 `.toString` 值
 - 数组：返回该数组的字符串形式，*e.g.* `[1, 2, 3] -> "1, 2, 3"`
- 使用 `Boolean()` 将输入的各种类型的值强制转换成布尔值
 - 除了以下五个值的转换结果为 `false`，其他的值全部为`true`：
 - `undefined`
 - `null`
 - `-0`或`+0`
 - `NaN`
 - 空字符串

自动类型转换

- 以强制转换为基础
- 遇到以下三种情况时，JavaScript会自动转换数据类型：
 - 不同类型的数据互相运算
 - 对非布尔值类型的数据求布尔值
 - 对非数值类型的值使用一元运算符（`+` 和 `-`）
- 规则：预期什么类型的值，就调用该类型的转换函数
- 自动转换为布尔类型：自动调用`Boolean`函数
- 自动转换为字符串：主要发生在字符串的加法运算时，先将复合类型的值转为原始类型的值，再将原始类型的值转为字符串
- 自动转换为数值：自动调用`Number`函数
 - 除了加法运算符 `+` 有可能把运算符转为字符串，其他运算符都会把运算符自动转成数值

null 和 undefined

异同

- `null`与`undefined`都可以表示“没有”，含义非常相似，在`if`语句中它们都会被自动转为`false`，相等运算符甚至直接报告两者相等
- `null` 是一个表示“空”的对象，转为数值时为`0`，即 此处不应该有值
- `undefined` 是一个表示“此处尚未定义”的值，转为数值时为`NaN`，即 缺少值

用法

- `null`：

- 作为函数的参数，表示该函数的参数为空
- 作为对象原型链的终点
- `undefined` :
 - 变量声明但未赋值
 - 函数参数未提供，此时参数值为 `undefined`
 - 对象没有复制的属性
 - 函数没有返回值时，默认返回 `undefined`

布尔值

布尔值代表“真”和“假”两个状态，“真”用关键字 `true` 表示，“假”用关键字 `false` 表示，布尔值只有这两个值

下列运算符会返回布尔值：

- 前置逻辑运算符： `!`
- 相等运算符： `===` , `!==` , `==` , `!=`
- 比较运算符： `>` , `>=` , `<` , `<=`

如果 JavaScript 预期某个位置应该是布尔值，会将该位置上现有的值自动转为布尔值。转换规则是除了下面六个值被转为 `false`，其他值都视为 `true`：

- `undefined`
- `null`
- `false`
- `0`
- `NaN`
- `""/''`（空字符串）

注：空对象、空数组都是 `true`

数值

类型

- 基于 IEEE 754 标准的双精度 64 位二进制格式的值，即 $(2)^{2^31}$ （唯一）
- 数也是以64位浮点数形式储存
- 除了浮点数外，JavaScript中还有一些带符号的值：
 - `+Infinity`：超过了 JavaScript 能表示的最大浮点数的数值，正无穷大（正数除以 0，或极大正数溢出）
 - `-Infinity`：比任何负数都要小的数值，负无穷大（负数除以0，或极小负数溢出）
 - `NaN`：非数值

相关属性/方法

- 属性：

- `Number.MAX_VALUE/MIN_VALUE`：可以表示的最大/小值
- `Number.MAX_SAFE_INTEGER/MIN_SAFE_INTEGER`：JavaScript 能够准确表示（不丢失精度）的最大/小整数（ 2^{53} ）
- 方法：
 - `Number.isSafeInteger()`：检查值是否在双精度浮点数的取值范围内，超出这个范围的数字不再安全

表示法

- 字面形式直接表示（4种进制，内部会自动转为十进制）
 - 十进制：无前导0
 - 八进制：`0o/0O` 开头，或有前导0且只用到0-7的数值
 - 十六进制：`0x/0X` 开头
 - 二进制：`0b/0B` 开头
- 科学计数法：底数 e 指数，即 底数 $\times$ 指数，下面情况自动将数值转为科学计数法表示：
 - 小数点前的数字多于21位
 - 小数点后的0多于5个

特殊数值

- 正负零
 - JavaScript 的64位浮点数之中，有一个二进制位是符号位。这意味着，任何一个数都有一个对应的负值，包括0
 - 几乎所有场合，正零和负零都会被当作正常的0，唯一有区别的场所是，+0或-0当作分母，返回的值是不相等的 $\Rightarrow$ `+/-Infinity`
- NaN：特殊值，表示“非数字”
 - 有字符串和数值进行运算时，会自动将字符串x转为数值，但是由于x不是数值，所以最后得到结果为NaN，表示它是“非数字”
 - 当发生错误时，数学函数的运算结果会出现NaN
- Infinity：表示无穷（有正负）
 - 用来表示两种场景：
 - 一个正的数值太大，或一个负的数值太小，无法表示
 - 非0数值除以0，得到 `Infinity`
 - VS. NaN：
 - `Infinity`大于一切数值（除了NaN），`-Infinity`小于一切数值（除了NaN）
 - `Infinity`与NaN比较，总是返回false
 - 运算：
 - 四则运算，符合无穷的数学计算规则
 - 0乘以Infinity，返回NaN
 - 0除以Infinity，返回0
 - Infinity除以0，返回Infinity
 - Infinity加上或乘以Infinity，返回的还是Infinity

- Infinity减去或除以Infinity，得到NaN
- Infinity与null计算时，null会转成0，等同于与0的计算
- Infinity与undefined计算，返回的都是NaN

与数值相关的全局方法

- parseInt：用于将字符串转为整数
 - 可以接受第二个参数表示被解析的值的进制（2到36），返回该值对应的十进制数
 - 默认是转十进制
 - 如果字符串的第一个字符不能转化为数字（后面跟着数字的正负号除外）等，返回NaN
 - 如果字符串头部有空格，空格会被自动去除
 - 如果parseInt的参数不是字符串，则会先转为字符串再转换
 - 字符串转为整数的时候，是一个个字符依次转换，如果遇到不能转为数字的字符，就不再继续进行下去，返回已经转好的部分
 - 如果字符串以 0x/0X 开头，会将其按照十六进制数解析
- parseFloat：用于将一个字符串转为浮点数
- isNaN：用来判断一个值是否为 NaN
 - 只对数值有效，如果传入其他值，会被先转成数值
 - 对于空数组和只有一个数值成员的数组返回false，原因是这些数组能被Number函数转成数值，使用isNaN之前，最好判断一下数据类型
 - 判断NaN更可靠的方法是：利用NaN是唯一一个不等于自身的值的这个特点，进行判断

```
function myIsNaN(value) {  
    return value != value;  
}
```

- isFinite：返回一个布尔值，表示某个值是否为正常的数值
 - 除了Infinity、-Infinity、NaN和undefined这几个值会返回false，isFinite对于其他的数值都会返回true

注意

- **由于浮点数不是精确的值，所以涉及小数的比较和运算要特别小心：**在计算机中，浮点数采用二进制来表示，并且使用有限的位数来存储它们，因此浮点数无法精确地表示所有实数。因为0.1和0.2这样的十进制小数无法完全准确地表示为二进制浮点数。在二进制中，只能使用有限的位数表示分数，而这些分数可能无法精确等于0.1、0.2或0.3。
- NaN 不是独立的数据类型，而是一个特殊数值，它的数据类型依然属于 Number（typeof 验证）
- NaN 不等于任何值，包括它本身
- NaN 在布尔运算时被当作 false
- NaN 与任何数（包括它自己）的运算，得到的都是 NaN

字符串

- 零个或多个排在一起的字符，放在单引号或双引号之中
- 字符串默认只能写在一行内，分成多行将会报错，如果长字符串必须分成多行，可以在每一行的尾部使用反斜杠
- 连接运算符 (+) 可以连接多个单行字符串，将长字符串拆成多行书写，输出的时候也是单行
- 反斜杠 \ 在字符串内有特殊含义，用来表示一些特殊字符，所以又称为转义符

•	<code>\0</code>	:	null (	<code>\u0000</code>	)
•	<code>\b</code>	:	后退键 (	<code>\u0008</code>	)
•	<code>\f</code>	:	换页符 (	<code>\u000C</code>	)
•	<code>\n</code>	:	换行符 (	<code>\u000A</code>	)
•	<code>\r</code>	:	回车键 (	<code>\u000D</code>	)
•	<code>\t</code>	:	制表符 (	<code>\u0009</code>	)
•	<code>\v</code>	:	垂直制表符 (	<code>\u000B</code>	)
•	<code>\'</code>	:	单引号 (	<code>\u0027</code>	)
•	<code>\"</code>	:	双引号 (	<code>\u0022</code>	)
•	<code>\\</code>	:	反斜杠 (	<code>\u005C</code>	)

- 可以被视为字符数组，因此可以使用数组的方括号运算符，用来返回某个位置的字符
 - 如果方括号中的数字超过字符串的长度，或者方括号中根本不是数字，则返回 `undefined`
 - 字符串内部的单个字符无法改变和增删，修改并没有生效，这些操作会默默地失败，**不会返回错误信息**
- `length` 属性：返回字符串的长度，该属性也是无法改变且不会报错的
- JavaScript 不仅以 Unicode 储存字符，还允许直接在程序中使用 Unicode 码点表示字符，即将字符写成 `\uxxxx` 的形式，其中 `xxxx` 代表该字符的 Unicode 码点
- 解析代码的时候，JavaScript 会自动识别一个字符是字面形式表示，还是 Unicode 形式表示。输出给用户的时候，所有字符都会转成字面形式
- Base64 相关方法 (2个):
 - `btoa()`：任意值转Base64
 - `atob()`：Base64转原值
 - 注：不适合非ASCII字符，否则报错。=> 解决：先进行URI编码再进行Base64编码，即加一个中间层，举例如下：

```
function b64Encode(str) {  
    return btoa(encodeURIComponent(str));  
}
```

```
function b64DEcode(str) {  
    return decodeURIComponent(atob(str));  
}
```

对象（核心概念 + 最重要的数据类型）

对象

- 对象就是一组“键值对”(key-value) 的集合，是一种**无序**的复合数据集合
 - key（键名/属性/property）：
 - 对象的所有键名都是字符串，加不加引号都可以
 - 如果键名是数值，会被自动转为字符串
 - 如果键名不符合标识名的条件且也不是数字，则必须加上引号，否则会报错
 - value：
 - 对象的键值可以是任何数据类型，如果一个属性的值为函数，通常把这个属性称为“方法”，它可以像函数那样调用
 - 如果属性的值还是对象，就形成了链式引用
- 对象的属性之间用逗号分隔，最后一个属性后面可以加逗号，也可以不加
- 对象的引用：
 - 如果不同的变量名指向同一个对象，那么它们都是这个对象的引用，也就是说指向同一个内存地址
 - 修改其中一个变量，会影响到其他所有变量
 - 如果取消某一个变量对于原对象的引用，不会影响到另一个变量对原对象的引用
 - 引用只局限于对象，如果两个变量指向同一个原子类型（数字、布尔、字符串）的值，变量这时都是值的拷贝
- 属性操作：
 - 属性的读取
 - 读取对象的属性，有两种方法：
 - 使用点运算符
 - 使用方括号运算符
 - 键名必须放在引号里面，否则会被当作变量处理
 - 方括号运算符内部可以使用表达式
 - 数字键名可以不加引号，会自动转成字符串
 - 数值键名不能使用点运算符（因为会被当成小数点），只能使用方括号运算符
 - 属性的赋值
 - 点运算符和方括号运算符，不仅可以用来读取值，还可以用来赋值
 - JavaScript 允许属性的“后绑定”，也就是说，可以在任意时刻新增属性，没必要在定义对象的时候，就定义好属性
 - 属性的查看
 - 使用 `Object.keys(obj)` 方法查看一个对象的所有属性

- 使用 `Object.values(obj)` 方法查看一个对象的所有属性值
- 属性的删除： `delete obj.attr`
 - 用于删除对象的属性，删除成功后返回true
 - 删除一个不存在的属性不报错，而且返回true，不能根据delete命令的结果判定某个属性是否存在
 - 只能删除对象本身的属性，无法删除继承的属性（如 `toString`）
- 属性是否存在： `in` 运算符
 - 用于检查对象是否包含某个属性（注意，检查的是键名，不是键值），如果包含就返回true，否则返回false
 - 它的左边是一个字符串，表示属性名，右边是一个对象
 - 不能识别哪些属性是对象自身的，哪些属性是继承的，可以使用对象的 `hasOwnProperty` 方法判断是否为对象自身的属性
 - e.g. `obj.hasOwnProperty('toString')`
- 属性的遍历： `for...in` 循环
 - 用来遍历一个对象的全部属性
 - e.g. `for (var key in obj) {...}`
 - 遍历的是对象所有可遍历（enumerable）的属性，会跳过不可遍历的属性
 - 不可遍历属性举例： `Object.prototype.hasOwnProperty`、
`Array.prototype.push`、`Array.prototype.pop`、
`Array.prototype.map`
 - 它不仅遍历对象自身的属性，还遍历继承的属性
 - 一般情况下，都是只想遍历对象自身的属性，所以使用的时候应该结合使用 `hasOwnProperty` 方法，在循环内部判断一下某个属性是否为对象自身的属性

```
for (var key in obj) {
  if(obj.hasOwnProperty(key)) {
    console.log(key);
  }
}
```

函数

- 一段可以反复调用的代码块。函数还能接受输入的参数，不同的参数会返回不同的值
- 声明：
 - 三种声明方式：
 - **利用 function 命令声明函数**
 - function命令后面是函数名，函数名后面是一对圆括号，里面是传入函数的参数，函数体放在大括号里面，这种叫做函数声明
 - **函数表达式**
 - 采用变量赋值的写法将一个函数赋值给变量，因为赋值语句的等号右侧只能放表达式，所以这个函数又称函数表达式
 - function命令后面不带有函数名时为匿名函数

- function命令后面带有函数名时，该函数名只在函数体内部有效，在函数体外部无效，有两个用处：
 - 可以在函数体内部调用自身
 - 方便除错：除错工具显示函数调用栈时，将显示函数名，而不再显示这里是一个匿名函数
- 函数的表达式需要在语句的结尾加上分号，表示语句结束
- **Function 构造函数**
 - 可以传递任意数量的参数给Function构造函数，只有最后一个参数会被当做函数体，如果只有一个参数，该参数就是函数体
 - 可以不使用new命令，返回结果完全一样
 - 不直观，很少用
- 如果同一个函数被多次声明，后面的声明就会覆盖前面的声明。由于**函数名提升现象**，前一次声明在任何时候都是无效的
- 调用：
 - 要使用圆括号运算符，圆括号之中可以加入函数的参数
 - 函数体内部的return语句表示返回。JavaScript引擎遇到return语句，就直接返回return后面的那个表达式的值，后面即使还有语句也不会得到执行
 - return语句不是必需的，如果没有的话，该函数就不返回任何值，或者说返回undefined
 - 递归：调用自身
- **函数是第一等公民：**
 - JavaScript 语言将函数看作一种值，与其它值（数值、字符串、布尔值等等）地位相同
 - 凡是可以使用值的地方，就能使用函数。可以把函数赋值给变量和对象的属性，也可以当作参数传入其他函数，或者作为函数的结果返回。函数只是一个可以执行的值，并无特殊之处
 - 由于函数与其他数据类型地位平等，所以在 JavaScript 语言中又称函数为第一等公民。
- 函数名的提升：
 - JavaScript 引擎将函数名视同变量名，所以采用function命令声明函数时，整个函数会像变量声明一样，被提升到代码头部，也即所谓变量提升（hoisting）
 - 如果先调用，再采用赋值语句定义函数，JavaScript 就会报错

<pre>>> f1(); var f1 = function (){};</pre> ⚠️ TypeError: f1 is not a function [详细了解]	=	<pre>>> var f1; f1(); f1 = function(){};</pre> ⚠️ TypeError: f1 is not a function [详细了解]
--	--------------------------------------	---

代码第二行，调用f1的时候，f1只是被声明了，还没有被赋值，所以会报错。

- 如果同时采用function命令和赋值语句声明同一个函数，最后总是采用赋值语句的定义。因为变量提升，所以函数声明会被提升到最开始，而在执行过程中，变量被赋值重新覆盖为赋值语句中的定义。

- 函数属性：
 - `name`：返回函数的名字
 - 2种情况：
 - 如果是通过变量赋值定义的函数，如果变量的值是一个匿名函数，那么 `name` 属性返回变量名
 - 如果是通过变量赋值定义的函数，如果变量的值是一个具名函数，那么 `name` 属性返回函数表达式的名字
 - 用处：获取参数函数的名字
 - `length`：函数预期传入的参数个数，即函数定义之中的参数个数
- 函数方法：
 - `toString`：返回一个字符串，内容是 函数的源码，包括函数内的注释
- 作用域：
 - 作用域（scope）指的是变量存在的范围
 - JavaScript的2种作用域（ES5规范）
 - 全局作用域：变量在整个程序中一直存在，所有地方都可以读取
 - 函数作用域：变量只在函数内部存在
 - 函数内部定义的变量，会在该作用域内覆盖同名全局变量
 - 对于var命令来说，局部变量只能在函数内部声明，在其他区块中声明一律都是全局变量，只有函数所在的区块会造成作用域变化
 - 函数内部的变量提升：与全局作用域一样，函数作用域内部也会产生变量提升现象，在函数内，var命令声明的变量，不管在什么位置，变量声明都会被提升到函数体的头部
 - 函数本身也是一个值，也有自己的作用域：函数的作用域与变量一样，就是其声明时所在的作用域，与其运行时所在的作用域无关
- 函数参数：
 - 不是必需的，JavaScript 允许省略参数
 - 省略的参数的值就变为undefined（参数个数不对也不会报错）
 - 没有办法只省略靠前的参数，而保留靠后的参数。如果一定要省略靠前的参数，只有显式传入undefined
 - 如果有同名的参数，则取最后出现的那个值，即使后面的没有值或被省略，也是以其为准
 - 传递方式：
 - **传值传递**：函数参数是原始类型的值（数值、字符串、布尔值）在函数体内修改参数值，不会影响到原始值
 - **传址传递**：函数参数是复合类型的值（数组、对象、其他函数）传入函数的是原始值的地址，因此在函数内部修改参数，将会影响到原始值
 - 如果函数内部修改的，不是参数对象的某个属性，而是把参数对象整个替换成另一个值，这时不会影响到原始值（形参为地址副本，整体改动为改变副本指向）
 - `arguments` 对象：

- 只在函数体内部才可以使用
- 包含了函数运行时的所有参数，可以在函数体内部读取所有参数
- 通过arguments对象的length属性，可以判断函数调用时到底带几个参数
- 正常模式下，arguments对象可以在运行时修改
- 严格模式（'use strict';）下arguments对象是一个只读对象，修改它是无效的，但不会报错
- 闭包：函数和声明该函数的词法环境的组合
 - 解决的问题：占用内存、存在变量污染问题
 - 特征：
 - 有函数嵌套
 - 内部函数引用外部作用域的变量参数
 - 返回值是函数
 - 创建一个对象函数，让其长期驻留
- e.g.

```
function fa() {
  let a = 10;

  function fb() {
    a--;
    console.log(a);
  }

  return fb;
}

var fm = fa();
fm(); // 实际上是fb，输出9
fm(); // 实际上是fb，输出8
```

- 核心原理：fa() 函数作用域中的变量本应被自动清理，但是由于返回值 fb 中需要调用 fa() 作用域当中的变量 a，因此JS引擎选择不清理这个 a。例子里闭包指的就是 a 被“闭合”在 fb 的词法作用域里，只要内部函数在外部被继续使用，并且仍然访问着外部函数的变量，就形成闭包
- 立即调用的函数表达式（IIFE）
 - 在 Javascript 中，圆括号（）是一种运算符，跟在函数名之后，表示调用该函数
 - 在 JavaScript 中，function这个关键字即可以当作语句，也可以当作表达式。为了避免解析上的歧义，JavaScript引擎规定，如果function关键字出现在行首，一律解释成语句
 - IIFE：
 - 增加括号不让function出现在行首，让引擎将其理解成一个表达式
 - (function(){...})(); or (function(){...})();

- 推广：任何让解释器以表达式来处理函数定义的方法，都能产生立即调用的效果，如 `, / ! / ~ / - / +`
- 本质：
 - `function` 关键字不出现在行首，让解释器认为是表达式
 - 函数定义后加上 `()`，调用函数

数组

- 定义：按次序排列的一组值。每个值 的位置都有编号（从0开始），整个数组用方括号表示。
- 说明：
 - 除了在定义时赋值，数组也可以先定义后赋值
 - 任何类型的数据，都可以放入数组（数值、对象、数组、 函数）
 - 如果数组的元素还是数组， 就形成了多维数组
- 本质：数组属于一种特殊的对象。`typeof`运算符会返回数组的类型是`object`，数组的特殊性体现在，它的键名是按次序排列的一组整数（数组的键名其实也是字符串）
- `length` 属性：返回数组的成员数量，可以通过修改`length`属性值随时增减数组的成员
 - 数组的数字键不需要连续，`length`属性的值总是比最大的那个整数键大1
 - 数组是一种动态的数据结构，可以随时增减数组的成员
 - `length`属性是可写的，当`length`属性设为大于数组个数时，读取新增的位置都会返回`undefined`
 - 可以通过修改`length`属性值为0来删除所有的数组成员
 - JavaScript 使用一个32位整数保存数组的元素个数，`length`属性的最大值就是 2^2
 - 由于数组本质上是一种对象，所以可以为数组添加属性，但是这不影响`length`属性的值 => 实际上并不算作数组的元素，而是作为对象的属性添加到数组对象中
- `in` 运算符：检查某个键名是否存在，如果数组的某个位置是空位，`in`运算符返回`false`
- 遍历：
 - `for...in`：不推荐使用（不仅会遍历数组所有数字键，还 会遍历非数字键）
 - `for / while`
 - 数组的 `forEach()` 方法：
 - 可以用来遍历数组中的每个元素，并对每个元素执行一个回调函数。
 - 参数是一个匿名函数，这个函数接受一个参数 `array`，表示当前正在遍历的数组元素。
 - 在 JavaScript 中，回调函数是一种特殊的函数，它作为参数传递给另一个函数， 并且在某个特定事件发生或条件满足时被调用执行。回调函数常见于异步编程，用于处理异步操作的结果或通知。当异步操作完成时，回调函数被调用，以便处理结果或执行其他逻辑。
 - 语法

```
array.forEach(function(currentValue, index, arr) {
    // 你的逻辑代码
});
```

- 空位：
 - 当数组的某个位置是空元素，即两个逗号之间没有任何值，我们称该数组存在空位，数组的空位不影响length属性
 - 如果最后一个元素后面有逗号，不会产生空位
 - 数组的空位是可以读取的，返回undefined
 - 使用delete命令删除一个数组成员，会形成空位，并且不会影响length属性
 - 数组的某个位置是空位，与某个位置是undefined，是不一样的：空位就是数组没有这个元素，所以不会被遍历到，而undefined则表示数组有这个元素，值是undefined，所以遍历不会跳过

运算符

算术运算符（10个）

算术运算符

- 加法运算符： `x + y`
- 减法运算符： `x - y`
- 乘法运算符： `x * y`
- 除法运算符： `x / y`
- 指数运算符： `x ** y`
 - 右结合，多个指数运算符连用时，先进行最右边的计算
- 余数运算符： `x % y`
 - 返回前一个运算符被后一个运算符除所得的余数
 - 运算结果的正负由**第一个**运算符的正负号决定
- 自增运算符： `++x` or `x++`，一元运算符，作用是将运算符首先转为数值，然后加上1或者减去1，会修改原始变量
- 自减运算符： `--x` or `x--`
- 数值运算符： `+x`
- 负数值运算符： `-x`

加法运算符

- 数字+数字
- 数字+布尔值：布尔值都会自动转成数值，然后再相加
- 字符串+字符串：将两个原字符串连接在一起返回一个新的字符串
- 非字符串+字符串：非字符串会转成字符串，再连接在一起
- 对象相加：如果运算符是对象，必须先转成原子类型的值（`obj.valueOf().toString()`），然后再相加

```
var obj = {p: 1};
obj + 2; // "[object Object]2"
```

数值运算符，负数值运算符

- 可以将任何值转为数值（与Number函数的作用相同），会返回一个新的值，而不会改变原始变量的值
- 空数组转换为字符串后是空字符串，再将空字符串转换为数字时会得到 0
- 空对象在转换为字符串时会变成 "[object Object]"，再进行这类运算会返回 NaN

比较运算符（8个）

非相等比较

- 基本规则：先看两个运算符是否都是字符串，如果是，就按照字典顺序比较，否则，则将两个运算符都转成数值，再比较数值的大小
- 字符串：JavaScript 引擎内部首先比较首字符的 Unicode 码点，如果相等，再比较第二个字符的 Unicode 码点，以此类推，由于所有字符都有 Unicode 码点，因此汉字也可以比较
- 非字符串：
 - 如果两个运算符都是原始类型的值，则是先转成数值再比较
 - 任何值（包括NaN本身）与NaN比较，返回的都是false
 - 如果运算符是对象，会转为原子类型的值（ValueOf + toString），再进行比较

相等比较（== / ===）

- 相等运算符（==）比较两个值是否相等，严格相等运算符（===）比较它们是否为“同一个值”
- 如果两个值不是同一类型，严格相等运算符直接返回false，而相等运算符会将它们转换成同一个类型，再用严格相等运算符进行比较
- 严格相等运算符（===）
 - 两个复合类型（对象、数组、函数）的数据比较时，不是比较它们的值是否相等，而是比较它们是否指向同一个地址
 - 对于两个对象的比较，严格相等运算符和相等运算符比较的是地址，而大于或小于运算符比较的是值
 - undefined 和 null 与自身严格相等
 - !== 的结果是 === 的相反值
- 相等运算符（==）：
 - 比较相同类型的数据时，与严格相等运算符完全一样
 - 比较不同类型的数据时，先将数据进行类型转换，然后再用严格相等运算符比较
 - 原始类型的值会转换成数值再进行比较
 - 对象（这里指广义的对象，包括数组和函数）与原始类型的值比较时，对象转换成原始类型的值，再进行比较
 - undefined 和 null 与其他类型的值比较时，结果都为false，它们互相比时结果为true
 - != 的结果是 == 的相反值

- 相等运算符隐藏的类型转换，会带来一些违反直觉的结果，很容易出错，建议使用严格相等

布尔运算符（4个）

取反：!

- 用于将布尔值变为相反值，对于非布尔值，取反会将其转为布尔值
- 除了以下六个值取反后为true，其他值都为false：
 - undefined
 - null
 - false
 - 0
 - NaN
 - ""/''

且：&&

- 运算规则（注意返回的是值，不是布尔值）：
 - 如果第一个运算符的布尔值为true，则返回第二个运算符的值，如果第一个运算符的布尔值为false，则直接返回第一个运算符的值，且不再对第二个运算符求值
 - 连用：返回第一个布尔值为false的表达式的值，如果所有表达式的布尔值都为true，则返回最后一个表达式的值

或：||

- 运算规则（注意返回的是值，不是布尔值）：
 - 如果第一个运算符的布尔值为false，则返回第二个运算符的值，如果第一个运算符的布尔值为true，则直接返回第一个运算符的值，且不再对第二个运算符求值
 - 连用：返回第一个布尔值为true的表达式的值，如果所有表达式的布尔值都为false，则返回最后一个表达式的值

三元：?:

- 三元条件运算符由问 ? 和 : 组成，分隔三个表达式，如果第一个表达式的布尔值为true，则返回第二个表达式的值，否则返回第三个表达式的值

二进制位运算符（7个）

- 或 (or)：|
- 与 (and)：&
- 否 (not)：~
- 异或 (xor)：^
- 左移 (left shift)：<<
- 右移 (right shift)：>>

- 无符号右移： >>>

标准库

URI编解码函数

函数

- `encodeURIComponent`：将URI中不合法的字符（如空格、特殊符号等）进行转义处理，而保留合法的字符不变，操作的是完整的URI
- `encodeURIComponent`：操作的是URI的组件，如果链接内包含一个回调地址，回调地址是另外一个URL，此时需要使用 `encodeURIComponent()` 对回调地址进行转码，这样URL中就不会出现多个 `http://`、`&` 这样的特殊字符，方便对回调地址进行处理。e.g.
`"https://www.baidu.com/s?returnURL=" +
encodeURIComponent("http://www.test.cn")`
- `decodeURI`：解码完整URI
- `decodeURIComponent`：解码URI组件

不合法/需要编码的字符

- 控制字符：ASCII码值在0到31之间
- 空格： `%20`
- 非ASCII码字符：进行URL编码，转为UTF-8编码的字符并使用百分号编码
- 特殊字符（部分符号需要视情况而定）：
%（百分号）&（和号）?（问号）/（斜杠）:（冒号）=（等号）
+（加号）;（分号），（逗号）@（小于符号）[]（方括号）#（哈希符号）
{ }（大括号）|（竖线）"（双引号）<（小于符号）>（大于符号）

字符串对象String

`match` 方法

- 用于确定原字符串是否匹配某个子字符串。匹配返回一个数组，包含了第一个匹配的详细信息，没有匹配返回空数组。
- 使用正则： `/匹配的子字符串/修饰符`
 - 修饰符：
 - `g`：全局匹配，查找字符串中所有符合条件的结果
 - `i`：忽略大小写匹配
 - e.g.

```
var str = "cat, bat, sat, fat.";
var pattern = /at/gi;

var matches = str.match(pattern);
```

```
console.log(matches);  
// ['at', 'at', 'at', 'at']
```

search 方法

- 用法基本等同于 `match`，但是返回值为匹配的第一个位置，如果没有找到匹配，则返回-1

replace 方法

- 用于替换匹配的子字符串，一般情况下只替换第一个匹配
- 若使用带有 `g` 修饰符的正则表达式，则替换所有

charAt 方法

- 返回指定位置的字符，参数是从0开始编号的位置
- 如果参数为负数，或大于等于字符串的长度，返回空字符串

concat 方法

- 用于连接两个字符串，返回一个新字符串，不改变原字符串
- 如果参数不是字符串，会将其先转为字符串，然后再连接

slice 方法

- 用于从原字符串取出子字符串并返回，不改变原字符串，第一个参数是子字符串的开始位置，第二个参数是子字符串的结束位置（不含该位置）
- 如果省略第二个参数，表示子字符串一直到字符串结束
- 如果参数是负值，表示从结尾开始倒数计算的位置，即该负值加上字符串长度
- 如果第一个参数大于第二个参数，返回一个空字符串

indexOf / lastIndexOf 方法

- 用于确定一个字符串在另一个字符串中第一次/倒数第一次出现的位置，返回结果是匹配开始的位置（区分大小写）。
- 如果返回-1，就表示不匹配

trim 方法

- 用于去除字符串两端的空格，返回一个新字符串，不改变原字符串
- 该方法去除的不仅是空格，还包括制表符（`\t`、`\v`）、换行符（`\n`）和回车符（`\r`）

toLowerCase / toUpperCase 方法

- 用于将一个字符串全部转为小/大写
- 返回一个新字符串，不改变原字符串

split 方法

- 按照给定规则分割字符串，返回一个由分割出来的子字符串组成的数组
- 如果分割规则为空字符串，则返回数组的成员是原字符串的每一个字符
- 如果省略参数，则返回数组的唯一成员就是原字符串
- 如果满足分割规则的两个部分紧邻着或位于开头结尾，则返回数组之中会有一个空字符串
- 可以接受第二个参数，限定返回数组的最大成员数

localeCompare 方法

- 用于比较两个字符串，返回一个整数
- 如果小于0，表示第一个字符串小于第二个字符串；如果等于0，表示两者相等；如果大于0，表示第一个字符串大于第二个字符串
- 特点：考虑自然语言的顺序，如 `B > a`

日期对象Date

介绍

- Date对象是 JavaScript 原生的时间库，它以1970年1月1日00:00:00作为时间的零点，可以表示的时间范围是前后各1亿天（单位为毫秒）
- Date对象可以作为普通函数直接调用，返回一个代表当前时间的字符串

Date的方法

- `Date.now`：返回当前时间距离时间零点的毫秒数
- `Date.parse`：用来解析日期字符串，返回该时间距离时间零点的毫秒数
- `Date.UTC`：接受年、月、日等变量作为参数，返回该时间距离时间零点的毫秒数

构造函数 Date()

- 对它使用new命令，会返回一个Date对象的实例。
- 如果不加参数，实例代表的就是当前时间
- 可以接受多种格式的参数，返回一个该参数对应的时间实例

日期运算

- 类型自动转换时，Date实例如果转为数值，则等于对应的毫秒数；如果转为字符串，则等于对应的日期字符串
- 两个日期**实例对象**进行减法运算时，返回的是它们间隔的毫秒数
- 两个日期**实例对象**进行加法运算时，返回的是两个字符串连接而成的新字符串

实例对象常见方法

- `to` 类：从Date对象返回一个表示指定时间的字符串
 - `toString()`：普通字符串
 - `toUTCString()`：UTC格式
 - `toLocaleString()`：本地时间格式，可用参数指定地区如 `zh-CN`

- **get 类**：获取Date对象的日期和时间
 - getTime()**：返回实例距离1970年1月1日 00:00:00的毫秒数，等同于valueOf方法
 - getDate()**：返回实例对象对应每个月的几号（从1开始）
 - getDay()**：返回星期几，星期日为0，星期一为1，以此类推
 - getFullYear()**：返回四位的年份
 - getMonth()**：返回月份（0表示1月，11表示12月）
 - getHours()**：返回小时（0-23）
 - getMilliseconds()**：返回毫秒（0-999）
 - getMinutes()**：返回分钟（0-59）
 - getSeconds()**：返回秒（0-59）
 - getTimezoneOffset()**：返回当前时间与 UTC 的时区差异，以分钟表示，返回结果考虑到了夏令时因素
- **set 类**：设置Date对象的日期和时间
 - setDate(date)**：设置实例对象对应的每个月的几号（1-31），返回改变后毫秒时间戳
 - setFullYear(year [, month, date])**：设置四位年份
 - setHours(hour [, min, sec, ms])**：设置小时（0-23）
 - setMilliseconds()**：设置毫秒（0-999）
 - setMinutes(min [, sec, ms])**：设置分钟（0-59）
 - setMonth(month [, date])**：设置月份（0-11）
 - setSeconds(sec [, ms])**：设置秒（0-59）
 - setTime(milliseconds)**：设置毫秒时间戳
- 注：
 - 凡是涉及到设置月份，都是从0开始算的，即0是1月，11是12月
 - 参数都会自动折算。以setDate为例，如果参数超过当月的最大天数，则向下一个月顺延，如果参数是负数，表示从上个月的最后一天开始减去的天数

数组对象Array

介绍

- Array是 JavaScript 的原生对象，同时也是一个构造函数，可以用它生成新的数组
- 构造函数输入不同参数，行为不一致：
 - 无参数：空数组
 - 单个正整数：表示返回的新数组的长度
 - 单个非数值：表示返回的新数组的成员

- 多个参数：表示返回的新数组的成员

方法：

- `isArray`：
 - 返回一个布尔值，表示参数是否为数组
 - 它可以弥补 `typeof` 运算符的不足（只能显示数组的类型是 `Object`）
- `push`：
 - 在数组的末端添加一个或多个元素，并返回添加新元素后的数组长度
 - 改变原数组
 - 配合 `pop` 可构成栈
- `pop`：
 - 用于删除数组的最后一个元素，并返回该元素，对空数组使用`pop`方法，不会报错，而是返回`undefined`
 - 改变原数组
 - 配合 `push` 可构成栈
- `shift`：
 - 用于删除数组的第一个元素，并返回该元素
 - 配合 `push` 可构成队列
- `unshift`：
 - 用于在数组的第一个位置添加元素，并返回添加新元素后的数组长度
- `join`：
 - 以指定参数作为分隔符，将所有数组成员连接为一个字符串返回
 - 如果不提供参数，默认用逗号分隔
 - 如果数组成员是`undefined`或`null`或空位，会被转成空字符串
- `concat`：
 - 用于多个数组的合并。它将新数组的成员，添加到原数组成员的后部，然后返回一个新数组，原数组不变
 - 除了数组作为参数，也接受其他类型的值作为参数，添加到目标数组尾部
- `reverse`：
 - 用于颠倒排列数组元素，返回改变后的数组
 - 注意，该方法将改变原数组
- `slice(start, end)`：
 - 用于提取目标数组的一部分，返回一个新数组，原数组不变
 - 第一个参数为起始位置（从0开始），第二个参数为终止位置（但该位置的元素本身不包括在内）
 - 如果省略所有参数，返回原数组的拷贝
 - 如果省略第二个参数，则一直返回到原数组的最后一个成员
 - 如果`slice`方法的参数是负数，则表示倒数的位置
 - 如果第一个参数大于等于数组长度，或者第二个参数小于第一个参数，则返回空数组
- `splice(start, count, addElement1, ...)`：

- 用于删除原数组的一部分成员，并可以在删除的位置添加新的数组成员，返回值是被删除的元素
- 注意，该方法会改变原数组
- 第一个参数为起始位置（从0开始），第二个参数为被删除的元素个数，如果后面还有更多的参数，则表示这些就是要被插入数组的新元素
- 起始位置是负数表示从倒数位置开始删除
- 第二个参数设为0表示单纯地插入元素
- 只提供第一个参数表示删除到结尾
- `sort` :
 - 对数组成员进行排序，默认是按照字典顺序排序。排序后，原数组将被改变
 - 数值会被先转成字符串，再按照字典顺序进行比较
- `map` :
 - 将数组的所有成员依次传入参数函数，然后把每一次的执行结果组成一个新数组返回，原数组没有变化
 - 参数为回调函数
 - 函数调用时，`map`方法向它传入三个参数：**当前成员、当前位置和数组本身**
 - 如果数组有空位，`map`方法的回调函数在这个位置不会执行，会跳过数组的空位，但不会跳过`undefined`和`null`
 - *e.g.*

```
[1, 2, 3].map(function(elem, index, arr) {
  arr[index] = 2 * elem;
  return;
});
// [2, 4, 6]
```

- `forEach` :
 - 用法与 `map` 一致，参数为回调函数，机制也相同
- `filter` :
 - 用于过滤数组成员，满足条件的成员组成一个新数组返回，该方法不会改变原数组
 - 它的参数是一个函数，函数有三个参数：当前成员，当前位置和整个数组
 - 所有数组成员依次执行该函数，返回结果为`true`的成员组成一个新数组返回
- `some()/every()` :
 - 这两个方法类似“断言”，返回一个布尔值，表示判断数组成员是否符合某种条件
 - 接受一个函数作为参数，所有数组成员依次执行该函数，该函数有三个参数：当前成员、当前位置和整个数组，然后返回一个布尔值
 - : `some` 方法判断是否存在符合条件的成员，只要一个成员的返回值是`true`，则整个`some`方法的返回值就是`true`，否则返回`false`
 - : `every` 方法判断是否所有成员都符合条件，所有成员的返回值都是`true`，整个`every`方法才返回`true`，否则返回`false`
 - 若只用到 `element`，则可以作为参数的函数只接受一个参数值
- `reduc()/reduceRight()` :

- 依次处理数组的每个成员，最终累计为一个值
- `reduce`是从左到右处理（从第一个成员到最后一个成员），`reduceRight`则是从右到左（从最后一个成员到第一个成员）
- 参数为函数，每次 `return` 的值座位下一个循环的累积值（回调函数的第一个参数）传进去
- `indexOf/lastIndexOf`：
 - 返回给定元素在数组中第一次出现的位置，如果没有出现则返回-1
 - `lastIndexOf`方法返回给定元素在数组中最后一次出现的位置，如果没有出现则返回-1
 - 方法都可以接受第二个参数，表示搜索的开始位置
 - 两个方法内部使用严格相等运算符进行比较

Math对象

介绍

- `Math`是 JavaScript 的原生对象，提供各种数学功能
- 该对象不是构造函数，不能生成实例，所有的属性和方法都必须在`Math`对象上调用
- `Math`对象的静态属性，提供以下数学常数，这些属性都是只读的，不能修改：

`Math.E` : 常数 e 。

`Math.LN2` : 2 的自然对数。

`Math.LN10` : 10 的自然对数。

`Math.LOG2E` : 以 2 为底的 e 的对数。

`Math.LOG10E` : 以 10 为底的 e 的对数。

`Math.PI` : 常数 π 。

`Math.SQRT1_2` : 0.5 的平方根。

`Math.SQRT2` : 2 的平方根。

- Math对象的静态方法

`Math.abs()` : 绝对值

`Math.ceil()` : 向上取整

`Math.floor()` : 向下取整

`Math.max()` : 最大值

`Math.min()` : 最小值

`Math.pow()` : 指数运算

`Math.sqrt()` : 平方根

`Math.log()` : 自然对数

`Math.exp()` : `e` 的指数

`Math.round()` : 四舍五入

`Math.random()` : 随机数

- `random()` : 返回 `(,)` 之间的一个伪随机数, 可以自己写任意范围随机数/随机整数

```
// 随机数
function getRandomArbitrary(min, max) {
    return Math.random() * (max - min) + min;
}

// 随机整数
function getRandomInt(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

- Math对象的三角函数方法:

`Math.sin()` : 返回参数的正弦 (参数为弧度值)

`Math.cos()` : 返回参数的余弦 (参数为弧度值)

`Math.tan()` : 返回参数的正切 (参数为弧度值)

`Math.asin()` : 返回参数的反正弦 (返回值为弧度值)

`Math.acos()` : 返回参数的反余弦 (返回值为弧度值)

`Math.atan()` : 返回参数的反正切 (返回值为弧度值)

DOM

文档对象模型DOM

地位

- 要实现页面的动态交互和效果，操作HTML是核心，操作HTML其实就是对DOM的操作。
- DOM提供了用程序来动态控制HTML的接口。

什么是DOM

- DOM是W3C的标准
- 一个用于HTML和XML文档的应用程序编程接口（API），它定义了文档的逻辑结构以及访问和 操作文档（改变文档的结构、样式和内容）的方式

提供的能力

- DOM允许程序和脚本动态地访问和更新文档的内容、结构和样式
- 使用文档对象模型（DOM），程序员可以构建文档、导航其结构以及添加、修改或删除元素和内容
- 除了少数例外，在HTML或XML文档中找到的任何东西都可以使用文档对象模型访问、更改、删除或添加
- 作为一个W3C规范，DOM的一个重要目标是提供一个可用于各种环境和应用程序的标准编程接口，DOM被设计成与特定编程语言相独立，DOM接口规范可以用各种语言实现

DOM

- 在HTML DOM中，所有事物都是节点，浏览器会根据DOM模型，将结构化文档解析成一系列的节点，再由这些节点组成一个树状结构（DOM Tree），所有的节点和最终的树状结构，都有规范的对外接口
 - 整个文档是一个文档节点
 - 每个HTML元素是元素节点
 - HTML元素内的文本是文本节点
 - 每个HTML属性是属性节点
 - 注释是注释节点
- DOM节点：DOM的最小组成单位叫做节点（node），文档的树形结构由各种不同类型的节点组成，每个节点是文档树的叶子
 - 节点类型：7种

Document : 整个文档树的顶层节点

DocumentType : doctype 标签 (比如 `<!DOCTYPE html>`)

Element : 网页的各种HTML标签 (比如 `<body>` 、 `<a>` 等)

Attribute : 网页元素的属性 (比如 `class="right"`)

Text : 标签之间或标签包含的文本

Comment : 注释

DocumentFragment : 文档的片段

- 元素节点：Element，元素节点是组成文档树的重要部分，它表示了HTML、XML文档中的元素。通常元素有子元素、文本节点或者两者的结合，元素节点

是唯一能够拥有属性的节点类型。html、head、title、body、div、ul、li等都属于Element

- 属性节点：Attr，属性节点代表了元素中的属性，因为属性实际上是附属于元素的，属性节点其实被看做是包含它的元素节点的一部分，它并不作为单独的一个节点在文档中出现。HTML元素的属性如id、class、href等都属于Attr（属性节点）
- 文本节点：Text，文本节点是只包含文本内容的节点，文本节点可以只包含空格。在文档树中元素的文本内容和属性的文本内容都是由文本节点来表示
- 注释节点：Comment，注解节点其实就是注释内容
- 文档节点：Document，文档节点是文档树的根节点，它是文档中其他所有节点的父节点。文档节点并不是HTML、XML文档的根元素，因为在XML文档中，处理指令、注释等内容可以出现在根元素之外，所以在构造DOM树的时候，根元素并不适合作为根节点，因此就有了文档节点，而根元素是作为文档节点的子节点出现的。
- 文档类型节点：DocumentType，每个Document都有一个DocumentType属性，它的值或是null，或者是DocumentType对象。比如声明文档类型时就是文档类型节点。
- 文档片段节点：DocumentFragment，文档片段节点是轻量级的或最小的Document对象，它表示文档的一部分或者是一段，不属于文档树。它的特殊性在于，当请求把一个DocumentFragment节点插入到文档的时候，插入的不是DocumentFragment自身，而是它的所有 的子孙节点。这使得DocumentFragment可以用来暂时存放那些一次插入文档 的节点，有利于实现文档的剪切、复制和粘贴等操作。
- DOM树：
 - 浏览器原生提供document节点代表整个文档
 - DOM树的第一层只有一个节点，它构成了树结构的根节点，其他HTML标签节点都是它的下级节点
 - 除了根节点和根元素，其他节点都有三种层级关系，DOM提供操作接口，用来获取这三种关系的节点
 - 父节点关系（parentNode）：直接的那个上级节点
 - 子节点关系（childNodes）：直接的下级节点
 - 兄弟节点关系（sibling）：拥有同一个父节点的节点

DOM节点相关接口

- DOM节点**Node**的主要属性、操作节点的常用操作
 - 所有DOM节点对象都继承了Node接口，拥有一些共同的属性和方法，这是DOM操作的基础

属性

- `nodeType`
- `nodeName`
- `nodeValue`
- `textContent`
- `baseURI`
- `ownerDocument`
- `nextSibling`
- `previousSibling`
- `parentNode`
- `parentElement`
- `firstChild`, `lastChild`
- `childNodes`
- `isConnected`

方法

- `appendChild()`
- `hasChildNodes()`
- `cloneNode()`
- `insertBefore()`
- `removeChild()`
- `replaceChild()`
- `contains()`
- `compareDocumentPosition()`
- `isEqualNode()`, `isSameNode()`
- `normalize()`
- `getRootNode()`

• 主要属性：

- `nodeType`：返回一个整数值，表示节点类型，Node对象定义了几个常量，对应这些类型值，用于确定节点类型

文档节点 (`document`) : 9, 对应常量 `Node.DOCUMENT_NODE`

元素节点 (`element`) : 1, 对应常量 `Node.ELEMENT_NODE`

属性节点 (`attr`) : 2, 对应常量 `Node.ATTRIBUTE_NODE`

文本节点 (`text`) : 3, 对应常量 `Node.TEXT_NODE`

文档片断节点 (`DocumentFragment`) : 11, 对应常量 `Node.DOCUMENT_FRAGMENT_NODE`

文档类型节点 (`DocumentType`) : 10, 对应常量 `Node.DOCUMENT_TYPE_NODE`

注释节点 (`Comment`) : 8, 对应常量 `Node.COMMENT_NODE`

- `nodeName`：返回节点的名称，不同节点的`nodeName`属性值如下：

文档节点 (`document`) : `#document`

元素节点 (`element`) : 大写的标签名

属性节点 (`attr`) : 属性的名称

文本节点 (`text`) : `#text`

文档片断节点 (`DocumentFragment`) : `#document-fragment`

文档类型节点 (`DocumentType`) : 文档的类型

注释节点 (`Comment`) : `#comment`

```
>> document.nodeName
<< "#document"
```

- `nodeValue`：

- 返回一个字符串，表示当前节点本身的文本值，该属性可读写。
- 只有文本节点 (`text`)、注释节点 (`comment`) 和属性节点 (`attr`) 有文本值，因此这三类节点的`nodeValue`可以返回结果/设置，其他类型的节点一律返回`null`/设置无效

- `textContent`：

- 返回当前节点和他的所有后代节点的文本内容。

- 对于文本节点 (text)、注释节点 (comment) 和属性节点 (attr)，textContent属性的值与nodeValue属性相同
- 对于其他类型的节点，该属性会将每个子节点（不包括注释节点）的内容连接在一起返回。如果一个节点没有子节点，则返回空字符串
- 可读写，设置该属性的值，会用一个新的文本节点替换所有原来的子节点
- `baseURI`：返回一个字符串，表示当前网页的绝对路径。浏览器根据这个属性，计算网页上的相对路径的URL。该属性为只读
- `ownerDocument`：返回当前节点所在的顶层文档对象，即document对象
- `nextSibling`：
 - 返回紧跟在当前节点后面的第一个同级节点，如果当前节点后面没有同级节点，则返回null
 - 该属性还包括文本节点和注释节点，如果当前节点后面有空格，该属性会返回一个文本节点，内容为空格
 - `nextSibling`属性可以用来遍历所有子节点（`firstChild + nextSibling`）
- `previousSibling`：返回当前节点前面的、距离最近的一个同级节点。如果当前节点前面没有同级节点，则返回null。和`nextSibling`属性一样，该属性还包括文本节点和注释节点。因此如果当前节点前面有空格，该属性会返回一个文本节点，内容为空格
- `parentNode`：
 - 返回当前节点的父节点
 - 对于一个节点来说，它的父节点只可能是三种类型：
 - 元素节点
 - 文档节点
 - 文档片段节点：DocumentFragment节点代表文档片段，是一个完整的DOM树形结构，它不属于当前文档，没有父节点，但可以插入任意数量的子节点，一般用于构建一个DOM结构，然后插入当前文档
- `parentElement`：返回当前节点的父元素节点，如果当前节点没有父节点，或者父节点类型不是元素节点，则返回null
- `firstChild/lastChild`：返回当前节点的第一个/最后一个子节点，返回的可能是元素节点、文本节点或注释节点，如果当前节点没有子节点，则返回null
- `childNodes`：返回一个类似数组的对象集合NodeList，成员包括当前节点的所有子节点
- `isConnected`：返回一个布尔值，表示当前节点是否在文档之中

```
var test = document.createElement("p");
test.isConnected; // false
document.body.appendChild(test);
test.isConnected; // true
```

- 常用操作（后续见PDF）
- DOM节点集合NodeList和HTMLCollection

- ParentNode和ChildNode接口

DOM中的主要对象

- Document节点对象
- 元素节点（Element）对象及属性、样式操作
- Text 节点和 DocumentFragment 节点

事件

题目整理

判断

1. 提交表单时使用 GET 方法时，表单数据在页面地址栏中是可见的。(T)
2. CSS的每一个样式声明之后，要用逗号表示一个声明的结束。(F)
3. 在JavaScript中，NaN === NaN表达式的返回值是true。(F)
4. 不同的HTML页面中可以出现相同id的元素。(T)
5. HTML标签大小写不敏感。(T)
6. Connection:keep-alive是HTTP/1.0请求的默认值。(F)
7. User-Agent首部的作用是让网络协议对端识别发起请求的用户代理软件的应用类型、操作系统、软件开发商以及版本号，以便显示不同的排版从而为用户提供更好的体验或者进行信息统计。(T)
8. CSS的样式声明用等号(=)分离开属性与属性值。(F)
9. CSS选择器中的class和id名称对大小写不敏感。(F)
10. 在JavaScript中，数组的length属性返回数组的成员数量，可以通过修改length属性值随时增减数组的成员。(T)

选择

1. 下面哪个HTML标签不能放到head标签内？(D)
 - A. meta标签
 - B. script标签
 - C. style标签
 - D. body标签
2. 利用 <iframe> 元素将另一个HTML页面嵌入到当前页面中时，利用（A）指向被嵌入的网页地址。
 - A. src="待嵌入页面地址"
 - B. href="待嵌入页面地址"
 - C. src:"待嵌入页面地址"
 - D. href:"待嵌入页面地址"
3. 跳转到“book.html”页面的“chapter1”指定位置的代码是（B）。
 - A. ...

- B. `<a href="book.html#chapter1">...</a>`
- C. `<a href="chapter1#book.html">...</a>`
- D. `<a href="chapter1&book.html">...</a>`

4. 下述哪个标签是HTML head标签中必需的标签？ (D)

- A. meta标签
- B. script标签
- C. style标签
- D. title标签

5. 同2

6. 有关HTML中的搜索引擎优化，下列说法**错误**的是 (D)。

- A. description被用来显示有关网页内容的信息，搜索引擎会在搜索引擎结果页面 (SERP) 使用它。
- B. robots用来阻止搜索引擎获取拷贝页面、私密页面和未完成的页面。
- C. title最重要，建议控制在70个字符以下，并在其中使用关键词。
- D. 在keywords标签中使用大量关键词有助于提高搜索引擎排名。

7. 有关CSS的层叠优先级算法，下列说法**错误**的是 (C)。

- A. 用户样式表中的普通声明优先级低于作者样式表中的普通声明
- B. 用户样式表中的重要声明优先级高于作者样式表中的重要声明
- C. 为了确保网页的设计保持预期，用户样式表中的样式声明优先级总是低于作者样式表中的样式声明
- D. 浏览器默认样式的优先级最低

8. 下列哪个选项的 CSS 语法是正确的？ (C)

- A. `body:color=black`
- B. `{body:color=black(body)}`
- C. `body {color: black}`
- D. `{body;color:black}`

9. 如果设定margin的CSS规则如下所示，则margin-bottom为 (C)。

```
div {margin: 5px 10px 15px 20px;}
```

- A. 5px
- B. 10px
- C. 15px
- D. 20px

10. 下列哪种方式是用类选择器定义样式的 (B)。

- A. `p {color: red;}`
- B. `.one {color: red;}`
- C. `#two {color: red;}`
- D. `p,h1{color: red;}`

11. CSS中， `padding:10px 20px 30px 40px` 代表的填充值顺序分别是 (A)。

- A. 上、右、下、左
- B. 上、下、左、右

- C. 上、左、下、右
- D. 左、右、上、下

12. 下列哪个是JavaScript中用来定义变量的? (A)

- A. var
- B. dim
- C. int
- D. char

13. 下列JavaScript方法返回结果正确是 (B)。

- A. 'hello world'.indexOf('o') 返回5 (4)
- B. hello world.indexOf('World') 返回-1
- C. hello world.indexOf('world') 返回7 (6)
- D. hello world.lastIndexOf('l') 返回2 (9)

14. JavaScript中, 表达式 123+'456' 的计算结果是 (A)。

- A. '123456'
- B. NaN
- C. '579'
- D. 579

15. JavaScript中, 表达式 '123abc'-'123' 的计算结果是 (B)。(减法只有数值可算)

- A. 'abc'
- B. NaN
- C. 123abc
- D. null

16. 同12

17. 下列JavaScript标识符是非法标识符的是 (D)。

- A. _temp
- B. \$e
- C. 临时变量
- D. 2a

18. 为链接设置样式时, 伪类的正确书写顺序是 (A)。

- A. a:link, a:visited, a:focus, a:hover, a:active
- B. a:link, a:focus, a:visited, a:hover, a:active
- C. a:link, a:focus, a:hover, a:active, a:visited
- D. a:link, a:active, a:hover, a:focus, a:visited

19. 以下选项中 (A) 是正确的URL。

- A. http://www.google.cn/index.html
- B. www.google.cn/greeting.html
- C. localhost:8080
- D. ftp.bupt.edu.cn

20. 有关HTTP协议, 下列说法中错误的是 (D)。

- A. HTTP是面向事务的应用层协议
- B. HTTP 1.0协议是无状态的
- C. HTTP协议本身是无连接的
- D. HTTP底层必须依赖面向连接的TCP提供的服务（HTTP/3基于UDP）

21. 下面是相对路径的是（C）。

- A. `https://www.bupt.edu.cn/img/logo2017.gif`
- B. `c:\index.html`
- C. `img/logo.gif`
- D. `ftp://202.108.221.2/pub/books/j.pdf`

22. 下列JavaScript标识符是合法标识符的是（C）。

- A. `sub-string`
- B. `this`
- C. `$param`
- D. `2a`

23. 有关JavaScript中的相等运算符，下列说法**错误**的是（A）。

- A. `NaN === NaN`返回true
- B. `null === undefined`返回true（先转成0再比）
- C. `{} === {}`返回false（对象比较地址）
- D. `[] === []`返回false（对象比较地址）

24. 有关HTTP状态码，下列说法正确的是（B）。

- A. 1xx：表示成功，如接受或知道了（2）
- B. 3xx：表示重定向，表示要完成请求还必须采取进一步的行动
- C. 5xx：表示客户的差错，如请求中有错误的语法（4）
- D. 4xx：表示服务器的差错，如服务器失效无法完成请求（5）

25. 下列元素中，为块级元素的是（C）

- A. `strong`
- B. `span`
- C. `h2`
- D. `input`

填空

1. HTTP状态码 **404** 代表请求资源不存在，如输入错误URL。
2. 在HTML中可以利用 **<form>** 元素插入表单。
3. CSS注释方法是 **/**/**
4. DNS解析流程分为 **递归查询** 和迭代查询两种。
5. **URL** 是对可以从因特网上访问的资源的位置和访问方法的一种简洁的表示。

主观

1. 简述表单的提交方法GET和POST之间的区别。

- 回答1: GET将表单数据附加在URL之后, 数据可见且长度受限, 安全性较低, 主要用于数据查询; 而POST将数据包含在HTTP请求体中, 隐蔽性好且理论上无长度限制, 更适合传输敏感信息或大量数据。
- 回答2: 如果没有敏感信息, 使用GET, 使用GET时, 表单数据在浏览器地址中是可见的, GET通过URL提交数据, 可提交的数据量和URL的长度有直接关系, HTTP协议规范对URL长度没有限制, URL长度受浏览器行WEB服务器限制, 包含敏感信息如密码使用POST, POST的安全性更好, 因为在浏览器地址栏中被提交的数据是不可见的, 实际开发中通常选择POST方式。

2. 简述如下HTTP请求头中各字段含义。

- 请求头:

```
GET / HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:83.0)
Gecko/20100101 Firefox/83.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;
q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-
US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
DNT: 1
Connection: keep-alive
Upgrade-Insecure-Requests: 1
If-Modified-Since: Thu, 17 Oct 2019 07:18:26 GMT
```

- 各字段含义:
 - **请求行 (GET / HTTP/1.1):** 请求使用 HTTP/1.1 协议的 GET 方法获取网站根目录资源。
 - **Host:** 指定被请求资源的 Internet 主机名和端口号, 此处指定请求的目标服务器域名为 `www.example.com`。
 - **User-Agent:** 包含发出请求的用户代理 (浏览器) 的信息, 如操作系统、浏览器内核、版本号等, 此处标识客户端身份为运行在 Windows 10 系统上的 Firefox 83.0 浏览器。
 - **Accept:** 告诉服务器客户端能够处理和接收的媒体类型 (MIME Type), 以及每种类型的优先级, 此处告知服务器客户端优先接收 HTML 页面, 同时也支持 XML 和 WebP 图片等格式。
 - **Accept-Language:** 告诉服务器客户端首选的自然语言, 以便服务器返回对应语言的页面, 此处告知服务器客户端首选语言为简体中文, 其次支持繁体中文和英语。
 - **Accept-Encoding:** 告诉服务器客户端支持的内容编码格式 (压缩算法), 用于减少传输数据量, 此处告知服务器客户端支持 `gzip` 和 `deflate` 两种内容压缩编码格式。
 - **DNT:** 全称 **Do Not Track** (请勿追踪)。这是用户隐私偏好设置, 此处值为 1 表示用户开启了“请勿追踪”功能, 请求服务器不要追踪用户行为。

- **Connection**: 控制网络连接的管理方式，决定在当前事务完成后是否关闭 TCP 连接，此处请求服务器在响应结束后保持 TCP 连接（长连接）以便后续请求复用。
- **Upgrade-Insecure-Requests**: 这是浏览器向服务器发送的一个信号，表示客户端支持并倾向于使用加密（HTTPS）和认证的响应，此处告知服务器客户端支持并希望将不安全的 HTTP 请求升级跳转为 HTTPS。
- **If-Modified-Since**: 这是**条件请求**首部，用于**缓存控制**，此处询问服务器自 2019 年 10 月 17 日后资源是否有更新，用于验证本地缓存是否有效。

3. 假设HTML页面的body源代码如下，编写代码，通过操作DOM修改网页CSS，按照步骤实现如下效果。

- 题目

```
<body>
  <div id="d0">
    <span class="hw s0"> Hello world 1!</span><br>
    <span class="hw s1"> Hello world 2!</span><br>
    <span id="s3"> Hello world 3!</span><br>
    <span id="s4"> Hello world 4!</span><br>
    <span id="s5"> Hello world 5!</span>
  </div>
</body>
```

1. 第一步：设定 div 下所有文本前景色都为 red；
2. 第二步：设定 Hello World 1! 和 Hello World 2! 的文本前景色为 green；
3. 第三步：利用CSS组合选择器设定 Hello World 3! 之后的所有span的前景色都为 orange

- 解答：

1. 第一步：

```
var div = document.getElementById("d0");
div.style.color = "red";
```

2. 第二步：

```
var greenSpans = document.querySelectorAll(".hw");
for (var i = 0; i < greenSpans.length; i++) {
  greenSpans[i].style.color = "green";
}
```

3. 第三步：

```
var orangeSpans = document.querySelectorAll("#s3 ~ span");
for (var i = 0; i < orangeSpans.length; i++) {
  orangeSpans[i].style.color = "orange";
}
```